

Abyssal Ascension

Rapport de projet

Table des matières

1. Introduction.....	5
1.1. Présentation du projet.....	5
1.2. Présentation de Caféine.....	6
2. Rappel du cahier des charges.....	8
2.1. Objectifs.....	8
2.2. Le protagoniste et les contrôles.....	8
2.3. Le monde et le level design.....	8
2.4. L'intelligence artificielle (les ennemis).....	9
2.5. Le système réseau (multijoueur).....	9
3. Conception.....	10
3.1. World Design.....	10
3.2. IA.....	13
3.3. Multijoueur.....	16
3.4. Menus & Interface utilisateurs.....	21
3.5. Textures & Animations.....	23
3.6. Son & Musique.....	27
3.7. Site Web.....	31
3.8. Contrôles.....	32
3.9. Système de sauvegarde.....	34
3.10. Système de fin de partie.....	35
3.11. Création de l'exécutable.....	36
4. Réalisation.....	38
4.1. Fonctionnalités jouables actuelles.....	38
4.2. Problèmes & Solutions.....	39
5. Synthèse des expériences individuelles.....	41
5.1. Elyas.....	41
5.2. Jonathan.....	42
5.3. Frédéric.....	43
5.4. Clément.....	45
6. Prévision.....	47
6.1. Accord avec le planning.....	47
6.2. World Design.....	48
6.3. IA.....	48
6.4. Multijoueur.....	49
6.5. Menus & Interface utilisateurs.....	49
6.6. Textures & Animations.....	49
6.7. Son & Musique.....	50

6.8. Site Web.....	50
6.9. Contrôles.....	51
6.10. Système de sauvegarde.....	51
6.11. Système de fin de partie.....	51
7. Conclusion.....	52
8. Annexes.....	53

1. Introduction

1.1. Présentation du projet

Abyssal Ascension est un jeu de plateforme 2D de type Metroidvania, inspiré de grands jeux tels que Hollow Knight et Ori. Le projet vise à proposer une expérience immersive mélangeant exploration, réflexion et progression dans notre univers.

Le jeu plonge le joueur dans un monde mystérieux, initialement sombre et oppressant, où la lumière constitue un élément central de la progression. Au fil de l'aventure, l'environnement s'éclaircit, cela symbolise à la fois l'ascension du personnage vers les hauteurs et donc son évolution au sein de cet univers.

Le joueur devra progresser à l'aide d'un nombre limité d'objets lui permettant d'avancer dans le jeu. Cette contrainte volontaire cherche à mettre l'accent sur la réflexion et l'observation de l'environnement plutôt que de juste monter en puissance.

L'exploration occupe une place centrale dans le jeu. Le monde est structuré de manière interconnectée pour encourager le joueur à revisiter des zones déjà explorées à l'aide de nouveaux objets découverts. Chaque zone possède sa propre identité visuelle et sonore afin d'obtenir un univers le plus riche et réaliste possible.

Le projet intègre également un système de jeu en multijoueur local. Ce système permet aux joueurs de coopérer ou d'interagir de différentes manières.

L'ensemble des graphismes, des sprites et des animations des personnages sont entièrement originaux afin de renforcer le visuel du projet.

De plus, nous pensons que la bande sonore joue un rôle important dans l'immersion du joueur. C'est pour cela que nos musiques originales et nos effets sonores fait maison ont été créés pour accompagner les différentes phases du jeu et renforcer l'atmosphère globale de notre jeu.

Abyssal Ascension est donc notre vision complètement originale du metroidvania, tel que nous l'imaginons : une expérience centrée sur l'exploration et la réflexion où l'ambiance du jeu est un élément clé pour l'immersion du joueur.

Nous n'avons pas choisi le metroidvania au hasard, ce type de jeu nous plaît particulièrement car il est différent des jeux "linéaires" classiques où on nous impose un chemin du début à la fin. Ici, le joueur a la liberté de choisir où aller et quand y aller.

Ce projet représente également pour nous une opportunité de mettre en pratique les compétences apprises lors de notre première année à l'EPITA, que ce soit en programmation, en organisation ou en rigueur. Abyssal Ascension est donc un jeu que nous avons imaginé et qui montre notre travail, notre créativité et notre capacité à créer une expérience complète.



1.2. Présentation de Caféine

Nous sommes Caféine, un groupe qui tire son nom de notre motivation pour ce projet et de notre addiction à la caféine (pas le café par contre, c'est pas bon). Ce nom montre à la fois notre énergie, notre implication et notre volonté de mener ce projet jusqu'au bout.

Notre équipe est composée de profils variés et complémentaires. Certains membres ont déjà eu l'occasion de participer à la création de jeux, ce qui nous apporte une première expérience dans le domaine. D'autres possèdent de bonnes bases en programmation, notamment en Python, tandis que certains se distinguent davantage par leur logique et leur capacité à résoudre des

problèmes. Cette diversité est un grand atout pour le développement de notre projet.

Ce mélange de compétences nous permet d'apprendre les uns des autres tout au long de la conception du jeu. Chacun peut apporter ses idées, partager ses connaissances et progresser dans des domaines qu'il maîtrise moins. Cela crée une dynamique de groupe où la collaboration est essentielle.

Nous répartissons les tâches selon les points forts de chacun, tout en restant flexibles. Nous cherchons à avancer efficacement tout en permettant à chaque membre de s'impliquer autant que possible.

Au cours du projet, nous avons aussi appris à mieux nous organiser entre nous. Au début, nous avons tendance à travailler chacun de notre côté sans forcément se concerter, ce qui pouvait créer des problèmes quand deux personnes modifiaient les mêmes parties du code. Petit à petit nous avons pris l'habitude de prévenir les autres avant de modifier une partie importante et de se donner des informations régulièrement sur l'avancée d'un point spécifique du projet.

Cette organisation nous a aussi appris à mieux estimer le temps que prend une tâche. Au début, on était souvent trop optimistes sur le temps des tâches et nous avons donc appris à prévoir une petite marge pour les imprévus.



2. Rappel du cahier des charges

2.1. Objectifs

Abyssal Ascension est un projet de jeu vidéo de plateforme 2D dans le genre Metroidvania. Il est inspiré de titres tels que *Hollow Knight* et *Ori*. Le projet a pour but d'intégrer un système de combat face à des ennemis intelligents dans un monde spécifique à l'univers d'Abyssal Ascension. Il inclut également l'intégration d'un mode multijoueur en LAN.

2.2. Le protagoniste et les contrôles

Le système de déplacement est le cœur du gameplay et repose sur une physique de vélocité pour garantir des mouvements fluides et naturels.

1. **Déplacements de base** : Course horizontale (touches Q/D ou flèches), saut et double saut.
2. **Mécaniques avancées** : Le dash pour une esquive rapide, le wall slide (glissade sur mur) et le wall jump (saut contre un mur) pour l'exploration verticale.
3. **Confort de jeu** : Intégration du coyote time pour accorder une courte tolérance de saut après avoir quitté une plateforme.
4. **Système de combat** : Une attaque au corps-à-corps (touche J ou clic gauche de la souris) avec un temps de recharge pour éviter le spam.
5. **Système de vie** : Une barre de points de vie (PV) gère la santé. La perte totale des PV déclenche une animation de mort avant la fin de partie.

2.3. Le monde et le level design

L'univers du jeu est structuré pour guider le joueur à travers la carte du monde d'Abyssal Ascension.

1. **Conception par tuiles (tileset)** : Le monde est dessiné bloc par bloc avec le logiciel LDtk.
2. **Système de portes** : Les zones sont reliées entre elles par des portes. Un système de décalage (offset) et une force verticale empêchent le joueur de rester bloqué ou de se téléporter en boucle entre les portes.
3. **Chargement dynamique** : Le jeu charge uniquement la pièce où se trouve le joueur pour optimiser les performances.

2.4. L'intelligence artificielle (les ennemis)

Les ennemis du jeu apportent de la difficulté et sont gérés par une machine à états.

1. **L'ennemi de base** : Il possède un comportement automatique divisé en plusieurs phases : patrouille autonome, champ de vision, état d'alerte, poursuite rapide et attaque au corps-à-corps.
2. **Le boss (Sire Radiantus Maximus)** : Un ennemi puissant doté d'attaques imprévisibles, dont un saut lourd au sol (slam) avec zone de dégâts et une attaque au corps-à-corps rapide.
3. **Changement de phase** : Si le boss a moins de 50% de sa vie, il devient plus rapide, agressif et imprévisible (phase 2), ce qui modifie le rythme du combat.

2.5. Le système réseau (multijoueur)

Le jeu permet à plusieurs joueurs de coopérer en réseau local.

1. **Architecture Client-Serveur** : Une machine héberge la partie (le serveur) et centralise les calculs de l'IA et du monde pour éviter les désynchronisations. Les autres machines (les clients) peuvent s'y connecter.
2. **Échange de données** : Les informations sont transmises au format JSON.
3. **Chargement des salles** : Le serveur calcule uniquement la physique des salles occupées, permettant aux joueurs de se séparer dans des pièces différentes sans réduire les performances du jeu.
4. **Gestion des fins de partie** :
 - a. Solo : Mort immédiate, écran "GAME OVER" et réinitialisation des ennemis.
 - b. Multijoueur : Délai de réapparition de 5 secondes pour le joueur mort sans bloquer la partie des autres joueurs.
 - c. Défaite totale (en multijoueur) : Si tous les joueurs meurent en même temps, la zone est entièrement remise à zéro.

3. Conception

3.1. World Design

Sans monde, il n'y a pas d'Abyssal Ascension. L'univers du jeu est le cœur de l'expérience. Il est donc impératif de consacrer une part importante du développement au world building.

Pour cela, nous utilisons le logiciel LDtk (Level Design ToolKit). Ce logiciel a été spécifiquement pensé et optimisé pour la conception de mondes 2D interconnectés des jeux de type metroidvania. Il nous offre une interface visuelle intuitive qui permet de concevoir l'univers de manière simple et extrêmement efficace, tout en générant des données facilement lisibles par notre code.

Pour le début de la conception du monde, on a choisi de prendre un design (tileset) simple composé de sol en pierre et d'une forêt dense en arrière-plan. Fidèle au jeu, plus le joueur progresse et monte dans les niveaux, plus la forêt en arrière-plan s'éclaircit visuellement (cf. Figure 2 et 3), cette transition lumineuse sert de repère visuel et récompense la progression du joueur.

Dans un jeu de type Metroidvania, le level design ne consiste pas uniquement à créer des plateformes ou des obstacles. Il doit aussi guider naturellement le joueur dans son exploration sans avoir recours à des indications explicites comme des flèches ou des objectifs affichés à l'écran. Nous avons donc cherché à utiliser l'environnement lui-même comme moyen de communication entre le jeu et le joueur.

Nous avons également réfléchi à comment les mécaniques de déplacement du joueur peuvent influencer la conception des niveaux. Une zone contenant de grands espaces verticaux encourage naturellement l'utilisation du wall jump tandis que des passages plus larges peuvent être pensés pour utiliser le dash. Ainsi, les mécaniques et les niveaux sont conçus en même temps afin que chaque élément du gameplay y trouve sa place.

Afin de gagner du temps lors du "level design", nous assemblons le décor bloc par bloc. Le design graphique est projeté sur un calque (layer) qui obéit à un système de règles d'auto-tiling pour pouvoir faciliter la conception des niveaux (cf. figure 7). Concrètement, nous définissons en amont comment les textures doivent se comporter. Lorsqu'on dessine une zone avec notre souris, le logiciel analyse le contexte et sait automatiquement quelle forme géométrique adopter (un coin, un sol plat, un bord de falaise) en fonction des blocs adjacents.

Lors de la création des zones, il nous suffit donc de sélectionner une couche de matériau (comme “stone” ou encore “dirt”) et de tracer la forme globale du terrain. Les règles régies précédemment génèrent instantanément les bons visuels (cf. figure 9). Ce processus automatisé rend la création très intuitive, nous épargne le placement manuel des cases une par une, et nous permet de nous concentrer sur la jouabilité et l’architecture des niveaux.

Pour garantir une cohérence visuelle et technique, nous avons standardisé les dimensions de notre monde. Chaque zone dans le monde a une dimension fixe de 960x480px. Cette zone est divisée en cases, chaque cases (tiles) dans la zone a une taille de 24px par 24px (cf. Figure 4).

Pour que le jeu tourne bien à 60 images par seconde, nous avons dû réfléchir à la manière de charger ces cases en mémoire. Si le programme essayait d’afficher toutes les cases de toutes les zones en même temps nous perdriions beaucoup en performances. Nous avons donc choisi de charger uniquement la zone où se trouve le joueur et dès que le joueur franchit une porte, il est téléporté aux coordonnées de sortie de la porte dans une autre zone pendant que la zone elle-même se charge. Cela nous donne une transition instantanée entre les zones avec de bonnes performances peu importe la taille finale de la carte du monde de notre jeu.

L’agencement des zones entre elles pour former la carte globale est également rendu très simple grâce à l’interface de la gestion du monde du logiciel (cf. Figure 5 et 8). Il s’agit d’un système de “glisser-déposer”, il suffit juste de prendre et glisser la zone à l’endroit que l’on souhaite.

La mécanique permettant de passer d’une zone à l’autre repose sur un système de “portes”. Les portes permettant de relier les niveaux entre eux fonctionnent de la même façon qu’une couche. On crée une sous-partie dans le layer avec la classe entities et on lui ajoute une valeur pour qu’il puisse agir dans tous les niveaux sans restriction. Pour relier deux niveaux, on place une entité “porte” dans la première zone et on lui attribue un identifiant ainsi qu’une valeur cible pointant vers la porte de la seconde zone. On fait de même dans le niveau de destination. La liaison entre les deux portes se fait alors automatiquement. Ce système supprime alors les restrictions physiques, une porte à droite d’un niveau peut très bien téléporter le joueur vers le plafond d’une autre zone très éloignée sur la carte globale, offrant une liberté totale dans la conception de notre jeu. Cela rend le processus bien plus intuitif et facile.

Une fois le monde dessiné, le logiciel permet l’exportation de la map en png et en JSON, un fichier de données brut. Le format JSON est constitué ainsi : la grande

sous-partie “Level” contient les différents niveaux créés, qui se segmentent ensuite en plusieurs propriétés techniques indispensables.

Chaque niveau possède d'abord un identifiant unique ainsi que des dimensions précises en pixels avec les champs “pxWid” pour la largeur et “pxHei” pour la hauteur, ce qui permet au script de définir les limites de la zone de jeu.

À l'intérieur de ces niveaux, on retrouve les “layerInstances”, qui représentent les différentes couches de la carte. La plus importante est la couche de collision, cette liste permet au code de traduire le dessin en une matrice de données logiques et compréhensible par la machine, où une valeur spécifique comme le chiffre 2 indique un obstacle infranchissable, tandis que le 0 correspond à une zone vide où le joueur peut se déplacer.

Parallèlement à la matrice de collision, les autoLayerTiles s'occupent de la partie visuelle en faisant correspondre chaque coordonnée de la grille aux textures sources du tileset. Elle indique au programme quelles coordonnées de la source graphique (le tileset) doivent être affichées à quel endroit sur l'écran.

Enfin, le fichier répertorie également les “entityInstances”, qui servent à répertorier la position de tous les objets dynamiques de la carte, c'est-à-dire les joueurs et les ennemis, et les points d'intérêt sur la carte. C'est ici que sont enregistrées les coordonnées “X” et “Y” précises du point d'apparition du joueur, le “PlayerSpawn”, ou encore les positions des ennemis.

L'ensemble de cette architecture de données est conçu pour être lu par le script `ldtk_loader.py`. Ce script extrait, nettoie et ne conserve en mémoire que les données essentielles au moteur de jeu, optimisant ainsi le temps de chargement et de charger la map dans le jeu.

L'intégration du système de transition par les portes, essentiel pour interconnecter les multiples zones du monde d'Abyssal Ascension, représente l'un des défis techniques les plus complexes du développement. La mécanique repose sur un transfert précis du joueur vers une instance de niveau spécifique à des coordonnées rigoureusement définies.

Dans nos premières versions, nous avons été confrontés à des boucles de téléportation infinies. Le joueur, à peine arrivé dans la nouvelle zone, se retrouvait exactement sur les coordonnées de la porte de retour. Cela déclenchait immédiatement la porte de retour et le joueur se retrouvait de nouveau dans la zone précédente. Pour pallier ce dysfonctionnement, nous avons implémenté un système de décalage dynamique (offset) dans la zone d'arrivée. Ce décalage est calculé selon le vecteur de mouvement du joueur, une

entrée latérale de gauche à droite entraîne un repositionnement automatique vers la droite par rapport au point de sortie, empêchant ainsi toute collision immédiate avec le trigger de la porte. Si l'offset réglait le problème pour les déplacements horizontaux, les transitions verticales (aller dans une zone supérieure à la zone actuelle) ont nécessité une attention particulière en raison de l'influence constante de la gravité. Lors d'une transition vers une zone supérieure, la pesanteur tendait à faire retomber le joueur instantanément dans la porte téléportant de nouveau le joueur dans la zone précédente avant qu'il n'ait pu faire quoi que ce soit.

La solution retenue consiste à appliquer une force propulsive verticale, calibrée sur la puissance d'un saut standard, dès que le joueur franchit une porte vers le haut. Cette force verticale annule temporairement l'effet de la chute et offre une fenêtre de réactivité nécessaire pour que l'utilisateur puisse initier une direction latérale avant que la physique du jeu ne reprenne son comportement habituel, garantissant ainsi des transitions fluides et un excellent confort de jeu.

3.2. IA

Après avoir bâti l'architecture et les décors du monde d'Abyssal Ascension, il est indispensable d'introduire un premier niveau de difficulté pour le joueur. Un monde vide, aussi beau soit-il, ne fait pas un jeu. C'est pourquoi nous avons développé une entité ennemie que le joueur va rencontrer lors de son exploration du monde d'Abyssal Ascension.

Lors de la précédente soutenance, cette entité se contentait de patrouiller, c'est-à-dire de faire des allers-retours basiques, sans réelle conscience. Son comportement a depuis été considérablement modifié, pour devenir plus intelligent. Il repose maintenant sur ce qu'on appelle une "machine à états". C'est-à-dire un système où l'ennemi se trouve à tout instant dans un état précis (patrouille, alerte, poursuite, attaque ou retour) et passe de l'un à l'autre en fonction des différentes conditions de l'environnement autour de lui. Cette architecture rend le comportement de l'ennemi plus crédible et plus simple à faire évoluer car le comportement de l'entité à chaque état est défini dans une fonction à part.

L'utilisation d'une machine à états possède également un avantage important au niveau du développement et de l'évolution future du projet. Chaque état étant séparé dans une logique à part, il devient possible d'ajouter facilement de nouveaux comportements sans modifier entièrement le comportement de l'entité existante.

Par exemple, il serait possible dans le futur d'ajouter un nouvel état correspondant à une fuite lorsque les points de vie de l'ennemi deviennent faibles, ou encore un état permettant à plusieurs ennemis de coopérer entre eux lorsqu'ils détectent simultanément un joueur.

En patrouille, l'entité se déplace horizontalement entre deux bornes définies par une position d'origine et un rayon de patrouille paramétrable (cf. figure 15). Deux conditions peuvent entraîner un changement de direction : le dépassement des limites de la zone de patrouille, ou la détection d'une collision avec un obstacle latéral. Cette seconde condition est importante car elle permet à l'ennemi de réagir à l'environnement. Ainsi, l'ennemi ne donne pas l'impression de courir dans le mur afin d'essayer de le traverser. Cela évite les bugs visuels et d'adopter un comportement réaliste rendant donc le jeu bien plus naturel.

A chaque frame (chaque image par seconde), l'entité vérifie également si le joueur entre dans son champ de vision. Ce champ est défini par une portée horizontale et une portée verticale, et la détection n'est validée que si le joueur se situe du côté vers laquelle l'ennemi est tourné. Le champ de vision n'est donc pas un simple cercle, mais plus comme un faisceau d'une lampe torche.

L'ennemi ne peut pas repérer un joueur situé dans son dos, ajoutant ainsi une nouvelle façon de jouer discrètement et de ne pas avoir à recourir au combat inutilement à chaque fois.

Dès lors que le joueur entre dans le champ de vision de l'ennemi, l'entité passe en état d'alerte : elle s'immobilise et se tourne vers sa cible. Cette courte pause est un choix de conception volontaire très connu dans le monde du jeu vidéo car elle donne au joueur un court instant visuel pour comprendre qu'il a été repéré et de se préparer au combat ou décider de prendre la fuite, juste avant que l'ennemi ne commence à se rapprocher et à attaquer le joueur.

Une fois que l'état d'alerte est terminé (temps écoulé), l'entité entre en poursuite, c'est-à-dire qu'elle se dirige vers le joueur à une vitesse nettement supérieure à celle de la simple patrouille, créant un véritable sentiment d'urgence. A partir de là, l'entité analyse en continu la distance le séparant du joueur, ainsi la situation peut évoluer de deux manières :

Si le joueur est assez rapide et s'éloigne assez, l'ennemi abandonne la poursuite et passe en état de retour qui le ramène progressivement vers sa position de départ avant de reprendre sa patrouille (si l'entité détecte un joueur pendant son état de retour elle peut passer directement en état d'alerte).

A l'inverse, si le joueur reste à portée et devient suffisamment proche alors l'entité passe en état d'attaque. Dans cet état, l'ennemi s'arrête et inflige des dégâts en continu au joueur tant que celui-ci reste trop près.

Pour que le combat ait un sens, il faut que le joueur puisse triompher. L'ennemi dispose donc également d'un système de points de vie, il peut donc subir des dégâts infligés par le joueur et disparaît (meurt) de la carte quand sa vie atteint zéro.

Pour l'étape suivante de notre projet, on s'est rendu compte que l'ennemi de base était un bon début, mais qu'il fallait un vrai défi marquant pour le joueur. On a donc développé un personnage bien plus fort et complexe, le Boss aussi nommé "Sire Radiantus Maximus". Techniquement, il réutilise la même structure de code en machine à états pour des raisons de gain de temps et de propreté, mais on a complètement modifié sa logique de décision pour le rendre beaucoup plus difficile et surtout moins prévisible.

Au tout début de nos tests, la machine à états de notre boss avait un gros problème : elle était trop simple à deviner. Dès qu'on s'approchait de lui, il lançait automatiquement son attaque au corps-à-corps. Il suffisait donc de faire une technique de "va-et-vient" (avancer pour déclencher l'attaque, reculer pour l'esquiver, puis revenir pour le frapper pendant son animation). Le combat devenait vite répétitif et sans intérêt. Pour pallier ce problème et forcer le joueur à rester attentif, on a implémenté trois gros changements dans son comportement. Tout d'abord, un système de temps de recharge. Désormais, quand le boss donne un coup d'épée, un compteur bloque cette attaque pendant un certain nombre de frames. Cela l'empêche de spammer le même coup et nous oblige, en tant que développeurs, à lui faire faire autre chose.

Puis, on a ajouté de l'aléatoire et des probabilités. Quand le joueur est dans sa zone de mêlée, le boss fait un choix dynamique. S'il peut attaquer à l'épée, il y a quand même un pourcentage de chances qu'il préfère lancer une autre attaque pour surprendre le joueur. Et si son épée est en train de recharger, au lieu de rester passif sans rien faire, le code lui donne une chance de lancer directement son attaque lourde pour punir le joueur qui pensait être en sécurité.

Enfin, on a implémenté une attaque sautée intelligente avec zone d'effet qu'on a appelé le slam. Le boss effectue un grand saut vertical avant de retomber lourdement. Pendant qu'il est en l'air, le script calcule l'écart horizontal entre le boss et le joueur. On applique alors une vitesse sur l'axe X pour que le boss ajuste sa trajectoire en plein vol. Il va traquer le joueur pour lui sauter dessus. Quand il touche le sol, il crée un grand rectangle de collision qui s'étend sur toute la zone sous lui, qui inflige des dégâts si le joueur se situe dans cette zone. Visuellement et techniquement, cela crée une grande onde de choc au sol, ce qui force le joueur à sauter avec le bon timing pour ne pas prendre de dégâts.

Pour finir et rendre le combat encore plus dynamique, on a implanté un système de Phase 2. Dès que la vie du Boss descend en dessous de 50 %, le code change ses variables statistiques. Sa vitesse de déplacement est multipliée par

1,5 pour poursuivre le joueur plus vite, et son temps de réflexion ainsi que le temps de recharge de son épée sont divisés par deux. Il devient extrêmement agressif, ce qui change le rythme du combat et ne laisse presque plus de temps mort au joueur pour respirer.

Pour que l'ennemi se déplace correctement dans le monde sans passer à travers le sol ou les murs, nous appliquons exactement la même logique physique que pour le personnage joueur. Plutôt que de calculer les déplacements diagonaux d'un seul coup, chaque axe est géré de manière indépendante et séquentielle, le jeu calcule d'abord si l'ennemi touche un obstacle horizontal et modifie la position si besoin, puis les collisions verticales sont traitées dans un second temps.

Cette méthode permet d'éviter les imprécisions liées aux collisions diagonales, qui peuvent survenir lorsque les deux axes sont traités simultanément. Les mouvements restent ainsi toujours fluides

La gravité de l'entité utilise la même variable que l'entité joueur afin d'obtenir un même comportement durant une chute. La gravité augmente la vitesse de chute de l'entité progressivement jusqu'à atteindre une vitesse maximale. Cette méthode permet de simuler l'effet de la pesanteur et d'assurer un comportement physique cohérent avec le reste du jeu.

3.3. Multijoueur

Un des piliers d'Abyssal Ascension est la coopération entre les joueurs. Nous avons implémenté une architecture client-serveur en LAN (Local Area Network) permettant une synchronisation fluide et cohérente entre deux instances du jeu tournant sur le même réseau local.

Nous avons choisi une architecture client-serveur plutôt qu'un système pair-à-pair (peer-to-peer). Dans une architecture pair-à-pair, chaque machine doit communiquer directement avec toutes les autres machines connectées, ce qui complexifie fortement la synchronisation et augmente les risques d'incohérence entre les états du jeu.

A l'inverse, avec une architecture client-serveur, une seule machine joue le rôle de "chef" : le serveur. Tous les calculs importants y sont effectués avant d'être redistribués aux clients. Cette approche permet de garantir que chaque joueur voit le même état du monde et simplifie énormément la gestion des ennemis, des collisions ou encore des dégâts.

Ce choix était particulièrement adapté à notre projet puisque le jeu se déroule sur un réseau local (LAN) avec un faible nombre de joueurs connectés simultanément.

Pour synchroniser les données en python, nous utilisons la bibliothèque native python `socket`. Le système tourne autour de 2 entités :

- Le serveur (host) : héberge la logique du jeu et distribue aux clients l'état du monde (positions de l'ensemble des joueurs, ennemis, statut de la partie...). Le serveur a toujours l'identifiant 0.
- Les clients : transmettent la position de leur joueur au serveur après l'application des inputs (entrées claviers / souris) permettant le déplacement du joueur.

Chaque client a un identifiant unique, il commence avec un identifiant temporaire de -1 (indiquant qu'il n'est pas encore connecté au serveur), puis une fois connecté, le serveur lui attribue un identifiant libre (par exemple 1 si ce client est le premier à se connecter sur le serveur). Cette méthode simplifie la gestion des données multijoueur puisque chaque paquet réseau peut être associé immédiatement au bon joueur sans problème.

(cf. figure 14)

L'échange des informations s'effectue via le format JSON. Ce format permet de structurer des données complexes (coordonnées, états, liste d'ennemis...) sous forme de chaînes de caractères lisibles.

Le choix de JSON comme format d'échange entre les machines a aussi été pratique pour le débogage. Quand un bug apparaît dans la communication réseau, nous pouvons facilement afficher le contenu des messages reçus dans la console pour vérifier que les données envoyées correspondent bien à ce qu'on attend. Si nous avions utilisé un format binaire plus compact, nous aurions sûrement eu un gain en performances mais nous aurions perdu beaucoup de temps à comprendre les problèmes de communication entre les différentes machines. Donc nous avons choisi de privilégier la lisibilité des données aux performances pour ce projet.

Le système réseau fonctionne selon une boucle continue exécutée pendant toute la durée de la partie. A chaque mise à jour réseau, chaque client envoie au serveur les informations importantes concernant son joueur local, notamment son identifiant, sa position, sa vitesse, sa vie, sa direction et ses différents états (animation, dash, sol...)

Le serveur réceptionne ensuite l'ensemble des données provenant des différents clients puis met à jour sa propre version du monde. Une fois les données traitées, il reconstruit un paquet contenant l'état complet de la partie avant de le transmettre à tous les joueurs connectés.

Cette architecture permet au serveur de rester la référence principale du monde du jeu. Les clients ne prennent donc jamais de décisions importantes concernant l'état global de la partie, ce qui évite de nombreuses incohérences.

Cette approche apporte également un avantage pour la cohérence du jeu. Si chaque client pouvait modifier directement les états importants du monde, des différences pourraient apparaître entre les différentes machines connectées. Deux joueurs pourraient par exemple voir un même ennemi à des positions différentes ou considérer des événements contradictoires comme valides.

La centralisation des décisions sur le serveur permet donc d'éviter ce type de situation. Le serveur agit comme une source unique de vérité : toute action importante doit être validée par celui-ci avant d'être propagée aux joueurs. Cette méthode est très fréquemment utilisée dans les jeux multijoueur modernes car elle simplifie la gestion des états complexes.

Pour maintenir une expérience multijoueur fluide, nous avons dû réfléchir à la fréquence de synchronisation des données réseau.

Lors des premières versions du système multijoueur, les données n'étaient synchronisées qu'une fois toutes les trois frames afin de limiter le nombre de paquets envoyés sur le réseau local. Cependant, cette méthode provoquait un problème visuel important : les déplacements des joueurs distants paraissaient saccadés et manquaient de fluidité.

Ce comportement venait du fait que les positions des joueurs étaient mises à jour trop peu fréquemment par rapport au rafraîchissement du jeu, qui fonctionne à 60 fps. Les autres joueurs semblaient donc "sauter" légèrement d'une position à une autre à l'écran.

Afin d'améliorer la fluidité visuelle, nous avons finalement choisi de synchroniser les données à chaque frame. Cela permet d'obtenir des déplacements beaucoup plus naturels et réactifs pour l'ensemble des joueurs connectés.

Même si cette méthode augmente légèrement le nombre de données transmises, elle reste largement acceptable dans le cadre d'un jeu en réseau local (LAN), où

la latence et les limitations de bande passante sont beaucoup moins importantes que sur internet.

Cependant, nous avons rencontré un problème majeur lors des tests du système multijoueur. Nous utilisons le protocole TCP (Transmission Control Protocol), et celui-ci ne garantit pas toujours la réception des messages sous la même forme qu'ils ont été envoyés. Par exemple, si on envoie le message "Bonjour" à une machine, celle-ci pourrait recevoir deux messages différents, le premier "Bon" et le second "jour". Cela faisait planter le système de chargement du JSON de la machine de réception car elle pouvait donc recevoir des messages JSON incomplets.

Pour résoudre ce problème, nous avons utilisé la méthode du préfixe de taille du message. Chaque message envoyé est précédé de la taille totale du message en bytes. Par exemple le message JSON suivant : `{"x":10,"y":20}` fait 16 bytes et sera donc envoyé sous la forme suivante : `[00000016][{"x":10,"y":20}]`, où les 4 premiers bytes contiennent la taille du message.

La machine qui reçoit les données lit d'abord les bytes reçus jusqu'à en obtenir 4, ce qui correspond à la taille du message. Elle continue ensuite la lecture jusqu'à avoir reçu l'intégralité des bytes correspondant au message.

Cette méthode permet donc à la machine de réception de reconstituer correctement le JSON, même si celui-ci a été découpé en plusieurs morceaux pendant la transmission.

Nous avons également intégré le changement de niveau dans le multijoueur. Lorsqu'un joueur prend une porte pour changer de zone, son niveau actuel est synchronisé avec les autres joueurs via le réseau. Ainsi, si deux joueurs se trouvent dans des niveaux différents, ils ne se voient plus à l'écran ce qui évite l'effet d'un joueur qui "vole" dans le vide au milieu d'une zone qui n'est pas la sienne.

Durant le développement, plusieurs problèmes de désynchronisation sont apparus. Dans certains cas, les ennemis ne se trouvaient pas à la même position selon les machines, ou bien un joueur pouvait sembler immobile pour certains clients alors qu'il continuait à se déplacer sur sa propre machine.

Ces problèmes provenaient principalement du fait que certaines données étaient encore calculées localement sur chaque machine au lieu d'être centralisées sur le serveur.

Nous avons progressivement corrigé ces incohérences en transférant davantage de responsabilités au serveur.

Depuis la précédente soutenance, nous avons élargi la synchronisation aux ennemis. Avant, chaque machine calculait le comportement des ennemis dans le niveau (coordonnées, état, vie,...) de son côté. Ce fonctionnement posait un problème de cohérence entre les machines (par exemple dans une machine tous les ennemis étaient regroupés à un endroit tandis que dans une autre machine tous les ennemis étaient regroupés à un autre endroit). Pour corriger cela, le serveur gère maintenant les ennemis comme les joueurs, il est la référence unique qui contient l'état de chaque ennemi de chaque zone. Il transmet ensuite l'état des ennemis en plus de celui des joueurs à l'ensemble des clients à chaque mise à jour réseau. Les clients se contentent ensuite d'afficher les ennemis aux positions reçues. Ainsi, tous les joueurs voient désormais exactement la même chose.

Lorsqu'un client attaque un ennemi en multijoueur, il n'applique pas les dégâts directement à l'ennemi mais il transmet l'information de son attaque au serveur. Le serveur applique alors les dégâts sur sa propre version des ennemis qui sera transmis à la prochaine synchronisation réseau.

L'implémentation de cette centralisation repose sur une gestion par environnement et sur la simulation de zones chargées par d'autres joueurs. Au début du jeu, le serveur extrait et stocke en mémoire les rectangles de collision (colliders) de l'ensemble des pièces, ce qui évite de les recalculer à chaque fois. Lors de la boucle principale de mise à jour, le serveur filtre la liste globale des joueurs pour savoir leur répartition dans les différents niveaux. Il exécute ensuite la physique et les scripts d'IA des ennemis uniquement pour les salles qui contiennent au moins un joueur. Les salles vides sont donc temporairement en pause. Cette architecture permet aux ennemis d'être synchronisés dans la zone d'un client, même si le joueur serveur se trouve dans une pièce différente. Le fait de charger les colliders à l'avance permet au serveur de continuer de tourner à 60 fps stable malgré le travail supplémentaire qu'il doit effectuer contrairement aux clients.

Nous avons également pris en compte la mort des joueurs dans le réseau. Lorsqu'un joueur meurt, il n'est plus affiché sur l'écran des autres joueurs, qu'il s'agisse du serveur ou d'un client. La synchronisation réseau continue cependant

de fonctionner après sa mort, afin que les autres joueurs ne soient pas affectés et que la partie se poursuive normalement pour eux. La gestion des cas où plusieurs joueurs ou bien la totalité d'entre eux meurent en même temps est expliquée dans la section 2.10 sur le système de fin de partie.

Pour finir, un problème important de notre système était la dépendance des clients par rapport au serveur. Au début, si le joueur qui hébergeait la partie (le serveur) fermait son jeu d'un coup, cela fermait brutalement la connexion de tous les joueurs. Le problème, c'est que ça faisait crash instantanément le jeu chez tous les autres joueurs connectés.

Pour éviter cela et rendre le jeu plus complet, nous avons ajouté une sécurité. Maintenant, si un client détecte que le serveur est déconnecté, le jeu gère l'erreur au lieu de planter complètement. A la place du crash, nous affichons un écran noir avec le message "Le joueur serveur a quitté". On a aussi mis un compte à rebours de 3 secondes. Dès que le temps est écoulé, le client nettoie ses variables réseau et renvoie directement le joueur sur le menu principal. De cette façon, même si le serveur part, les clients quittent la partie correctement sans que leur jeu ne s'arrête de force.

3.4. Menus & Interface utilisateurs

Le menu principal a pour image de fond, une image sombre représentant l'ambiance du début du jeu (les abysses).

Le thème principal est également joué en arrière-plan pour rendre le menu principal plus complet et vivant.

Pour faciliter la connexion avec des amis, on a décidé d'afficher l'adresse IP locale de la machine dans le menu principal. Cela permet de la partager et de pouvoir héberger puis rejoindre une partie multijoueur plus rapidement.

Trois options de jeu sont proposées :

- Solo : Lancement immédiat d'une partie locale.
- Héberger (multijoueur) : Création d'un serveur et lancement immédiat de la partie hébergée.
- Rejoindre (multijoueur) : Redirige vers une interface de saisie.

Dans le menu "Rejoindre", le joueur saisit l'IP du serveur à rejoindre. Un indicateur visuel valide la saisie en temps réel selon le format standard (présence des points et chiffres compris entre 0 et 255).

Le joueur peut également aller dans le menu "Paramètres", dans celui-ci il peut modifier le niveau de volume de la musique (entre 0% et 100%) et changer la résolution du jeu.

Lors de la conception de l'interface utilisateur, nous avons cherché à appliquer plusieurs principes afin que la navigation reste simple et intuitive. L'objectif principal était de limiter le nombre d'actions nécessaires pour accéder rapidement à une partie.

Nous avons aussi fait attention aux informations affichées à l'écran afin d'éviter une surcharge visuelle. Les éléments importants comme la vie ou certains paramètres essentiels doivent rester immédiatement visibles tandis que les informations davantage destinées au développement, comme les données de débogage restent facultatives.

Le joueur peut appuyer sur la touche "Echap" afin d'ouvrir un menu de pause. Depuis ce menu, le joueur peut effectuer diverses actions, le joueur peut :

- Reprendre la partie (ferme le menu de pause)
- Changer la résolution (le comportement de changement de résolution est décrit plus précisément dans la suite de cette partie)
- Afficher les données de débogage (affiche diverses informations sur l'état du jeu et du joueur en haut à gauche de l'écran)
- Afficher les boîtes de collisions (hitbox des joueurs, ennemis et portes de liaisons de niveaux)
- Afficher les nametags (noms en dessous des joueurs, seulement disponible en multijoueur)
- Contrôler le niveau de volume de la musique de fond ambiante, entre 0 et 100%
- Contrôler le niveau de volume des effets sonores (sfx), entre 0 et 100%
- Quitter le jeu

Lorsque le joueur se retrouve en jeu (en mode solo ou multijoueur) et qu'il active l'affiche des informations de debug, il peut voir de nombreuses informations de débogage en haut à gauche de son écran. Ces informations correspondent au mode de jeu actuel, au nombre de joueurs connectés sur le serveur, à l'identifiant du joueur, au nombre de FPS, à la position et à la vitesse du joueur local, ainsi qu'à sa vie et à son statut (au sol / en l'air).

De la même manière, si le joueur active l'affichage des boîtes de collision, l'ensemble des hitbox des différentes entités devient visible à l'écran.

Si le joueur décide de modifier la résolution, il peut choisir parmi plusieurs résolutions au format 2:1 : 960x480, 1280x640, 1600x800, 1920x960 ou encore le mode plein écran.

En mode plein écran, le jeu ajoute automatiquement des bandes noires au-dessus et en dessous lorsque cela est nécessaire afin de conserver le ratio d'origine sans déformation.

Cette technique est appelée "letterboxing" et elle est particulièrement utile puisque la majorité des écrans de nos jours utilisent un format 16:9 ou 16:10.

Le jeu remet ensuite l'entièreté des éléments visuels à l'échelle, pour ne pas avoir d'éléments en dehors de la fenêtre du jeu (un bouton rendu en dehors de la zone visible par exemple).

Lorsqu'un joueur meurt, l'écran de "GAME OVER" s'affiche sur son écran et il peut appuyer sur la touche R pour recommencer. Si il est en multijoueur, un délai est alors ajouté à la réapparition, après sa mort, le joueur doit attendre 5 secondes avant de pouvoir réapparaître. Le comportement de fin de jeu est décrit plus précisément dans la partie 2.10 "Système de fin de partie".

3.5. Textures & Animations

Pour tout ce qui touche à l'animation, nous avons choisi le style "pixel art".

Ce style graphique présente plusieurs avantages techniques dans le cadre de notre projet. Premièrement, il permet de produire des graphismes cohérents avec des ressources limitées tout en conservant une bonne identité visuelle. Deuxièmement, il facilite l'intégration des animations dans un jeu de plateforme 2D. Les mouvements peuvent être représentés avec relativement peu d'images tout en restant lisibles à l'écran. Cela permet également de réduire la taille des ressources utilisées par le jeu.

Nous avons aussi cherché à conserver une cohérence visuelle globale dans le choix des couleurs utilisées. Les environnements utilisent principalement des couleurs relativement sombres au début de l'aventure afin d'accentuer l'ambiance pesante et mystérieuse du jeu. Les personnages et ennemis utilisent quant à eux des contrastes légèrement plus marqués afin qu'ils restent facilement visibles pour le joueur.

De plus, chaque animation et graphique a été dessiné à la main. Le site internet que nos designers utilisent actuellement est "Piskel". L'interface est rapide à prendre en main et très pratique.

Pour ce faire, il faut suivre plusieurs étapes. Sur le logiciel de création utilisé, on

doit dans un premier temps dessiner ce que l'on appelle des "sprites" (cf. figure 10). Ils correspondent à une image unique. Via le logiciel, on va ensuite disposer plusieurs sprites à la suite et les faire défiler pour créer du mouvement (cf. figure 11). Cela permet alors d'importer chaque animation dans le jeu vidéo et de les lier à chaque action. Les graphismes présents sur l'écran d'accueil, les futures cinématiques ou encore le site internet ont également été créés via ces techniques.

Le pixel art a beau être un style "simple" en apparence, en réalité il demande beaucoup de travail sur chaque pixel. Contrairement à un dessin classique où une petite imperfection passe facilement inaperçue, un pixel mal placé dans un sprite peut totalement casser la lisibilité d'une animation. Nous avons donc passé beaucoup de temps à ajuster les sprites pixel par pixel pour que les mouvements paraissent naturels.

Pour l'implémentation dans le jeu, il n'y aura rien de très compliqué. Lors de l'écriture du code, nous avons des variables globales qui définissent le statut du joueur (course / marche / chute...). Lorsque l'on veut déclencher une animation spécifique, il suffira de relier les sprites correspondants au bon moment en fonction de la du statut du joueur. Tant que la condition est remplie, on fait défiler les sprites correspondants. Le personnage avance donc dans le jeu et l'animation tourne en boucle jusqu'à ce que ce dernier s'arrête.

Pour la suite, on ajoute les sprites directement dans un fichier du jeu prévu pour l'animation afin de pouvoir les afficher selon certaines conditions. Les images sont enregistrées en fichier .PNG afin de faciliter leurs utilisations. Chaque image est renommée d'une façon précise : 0-Running-0 signifie dans l'ordre qu'il s'agit du personnage N°1, de l'animation de course (running), du sprite N°0. Cela permet de s'y retrouver très facilement lorsque nous sommes amenés à coder pour les animations. Il faut aller chercher la bonne image au bon endroit. (cf. figure 12)

Pour rendre les animations de déplacements plus fluides, nous nous sommes basés sur de courts extraits de vidéos ou de films pour pouvoir décortiquer le mouvement dans son ensemble. Il nous a ensuite suffi de visionner ces extraits encore et encore afin de sélectionner les meilleures images, pour pouvoir les reproduire nous-mêmes à la main. Lorsque l'animation en question n'est autre qu'un mouvement perpétuel, il suffit alors de faire tourner une boucle dans le code qui passe en continu les mêmes images, afin de donner l'impression que le mouvement est continu.

Créer l'animation de course a été un vrai défi : il fallait que le personnage donne l'impression de courir vite sans avoir l'air de glisser sur le sol. En observant des

vidéos, on a remarqué comment le corps bouge et on a essayé de reproduire cela dans nos animations.

Pour l'animation d'attaque, le fonctionnement est différent car ce n'est pas une boucle continue, c'est un coup unique. Pour que le coup ait de la force à l'écran, nous avons essayé de découper l'action en trois étapes. D'abord le personnage recule un peu son arme (c'est l'élan) et ensuite il donne le coup. Nous avons eu énormément de mal à réaliser cette animation car il s'agit d'une animation qui fait bouger beaucoup de parties du corps du personnage. Nous avons donc dû réessayer des dizaines de fois avant d'obtenir un résultat qui nous paraissait correct. Même après toutes ces tentatives, nous ne sommes toujours pas complètement satisfaits du résultat car nous trouvons que le personnage est trop "rigide" durant son animation d'attaque et que la force de son attaque ne se ressent pas énormément.

Une autre animation spéciale est l'animation de mort. Celle-ci se déclenche seulement lorsqu'un joueur perd l'entière de sa vie. L'animation est jouée pendant trois secondes avant l'affichage du menu de fin de partie (Game Over). Il s'agit d'une animation spéciale composée de quatre images, les deux premières images représentent le personnage qui s'écroule sur le sol et les deux dernières images représentent le personnage qui se décompose en particules volant dans les airs. Ces deux dernières images sont des images spéciales qui sont alternées en boucle pour créer un effet "volatile" du personnage se décomposant dans les airs.

Nous avons appliqué différentes méthodes d'animation pour tous les autres mouvements du jeu nécessitant un indicateur visuel plus ou moins important sur le joueur, ces mouvements sont :

- Le saut : Nous avons créé deux images différentes pour l'impulsion de départ du joueur quand il commence à sauter.
- La chute : Nous avons créé trois images qui bouclent lorsque le joueur chute avec des petites particules grises qui défilent pour montrer la vitesse de la chute, cela permet d'ajouter du dynamisme à l'animation et amplifier l'effet de chute.
- Le dash : Comme c'est un déplacement ultra rapide, cette animation n'a que trois images qui ne changent que très légèrement et qui bouclent pendant le dash.
- Le wall slide (glissade sur le mur) : Ici, le personnage reste dans la même position. Il n'y a que deux images pour cette animation où seules des petites particules grises se déplacent pour montrer le mouvement vertical du joueur.

Pour afficher ces animations, il y a plusieurs conventions à respecter. Deux animations différentes ne peuvent pas avoir lieu en même temps. En effet, si le joueur est dans l'état initié "en saut" mais qu'il continue à avancer vers l'avant, l'animation qui doit être affichée est celle du saut et non celle de course. Il y a donc des priorités à déterminer lorsque l'on parle d'affichage d'animations. De plus, les obstacles doivent empêcher les animations d'avoir lieu. Si le joueur se heurte à un mur, sa course doit naturellement s'arrêter. Il y a par conséquent des conditions supplémentaires à implémenter pour toutes les interactions avec l'environnement. Des déplacements spécifiques tels que le sliding sur les murs sont aussi à prendre en compte lors de la création de ces règles.

Par la suite, implémenter l'affichage des animations n'est pas compliqué. Les états prédéfinis lors de la création de la classe "Personnage" servent à détecter les mouvements du joueur. Lorsque l'état qui nous intéresse est actif, il suffit de déclencher au même moment une boucle qui chargera à l'emplacement du joueur trois à quatre images à la suite. Ces images auront un temps d'apparition différent en fonction du type de mouvement. Pour la course par exemple, chaque sprite sera affiché durant 5 frames. Le jeu tournant à 60 FPS en permanence, cela permet de bien décortiquer le mouvement et de voir toutes les étapes de l'animation apparaître.

Pour optimiser le chargement des images sous Pygame, nous n'ouvrons pas les fichiers .PNG un par un pendant que le joueur court, faire cela ralentirait énormément le jeu à cause des accès constants au disque dur. A la place nous chargeons à l'avance l'ensemble des sprites au lancement du jeu. Toutes les images de toutes les animations sont chargées d'un coup dans des listes Python. De plus, pour que les sprites se retournent lorsque le joueur change de direction, nous utilisons la fonction `pygame.transform.flip()`. Cela nous évite de devoir dessiner deux fois chaque animation (une version vers la droite et une version vers la gauche), ce qui nous a fait gagner beaucoup de temps et a divisé par deux le nombre d'images à stocker.

La fluidité visuelle des animations repose également sur la gestion du rafraîchissement des frames d'animation par rapport au framerate du jeu. Faire défiler les sprites à chaque image du jeu (à 60 FPS) rendrait les mouvements beaucoup trop rapides et incompréhensibles. Pour régler ce problème, chaque état d'animation possède un compteur de temps qui permet de rajouter du délai entre chaque image de l'animation. Ce compteur détermine le nombre d'images du jeu pendant lesquelles un même sprite reste affiché à l'écran avant de passer au suivant dans la liste. Grâce à ce délai, nous pouvons donc ajuster l'impression de vitesse, par exemple une animation de course demande un défilement rapide alors qu'une animation de mort demande un défilement lent pour donner un effet

plus dramatique au personnage (une personne morte n'est pas censée se déplacer vite).

Pour tout ce qui est des monstres : le jeu Abyssal Ascension a comme ambiance une grotte plutôt sombre et mystérieuse. Les monstres à choisir pour affronter le joueur doivent donc être en accord avec le type du jeu ainsi que l'ambiance. Nous avons donc opté pour des zombies pour tout ce qui est au corps à corps, des squelettes pour du potentiel combat à distance (cela reste à déterminer lors de la dernière étape de création du jeu). Des monstres volants seront aussi implémentés pour l'aspect de déplacement. En effet, cela ajoute une dimension importante au jeu. Le joueur ne pourra pas qu'esquiver afin de passer aux salles suivantes. Il sera donc contraint d'apprendre les mécaniques de déplacements ainsi que d'esquive grâce à ces monstres particuliers. Les animations d'attaques des monstres ont été sauvegardées dans un fichier spécifique du jeu, mais n'ont pas encore été implémentées car le système de point de vie et d'attaque est en cours d'implémentation.

L'étape suivante est le design du second plan du jeu. L'affichage du world building fonctionne grâce à des tuiles. Ces dernières sont des planches sur lesquelles sont dessinées un environnement simple contenant quelques obstacles permettant de faire des tests facilement lors des implémentations. De la même manière que pour les animations de monstres ou du personnage principal, il suffit de dessiner des sprites assez grands pour pouvoir les superposer sur les tiles. Ces sprites ne changeront jamais car l'environnement reste fixe. Les tiles changeront en fonction du passage du joueur dans des salles différentes, mais chaque objet aura son propre sprite qui lui sera attribué, donc chaque changement d'environnement sera fait automatiquement.

3.6. Son & Musique

Dans un jeu de type Metroidvania, l'ambiance sonore occupe une place importante. Une grande partie de l'expérience repose sur l'exploration, la découverte et l'immersion du joueur dans un univers souvent calme et mystérieux.

La musique ne sert donc pas uniquement d'accompagnement, elle participe directement à la construction de l'atmosphère du jeu. Une musique trop dynamique ou trop répétitive risquerait par exemple de casser l'impression d'exploration et de solitude que nous cherchons à faire ressentir dans Abyssal Ascension.

Nous avons donc décidé de créer notre propre musique pour le jeu afin de renforcer l'immersion du joueur. Plutôt que d'utiliser des ressources externes, nous avons fait le choix de produire une bande sonore entièrement originale, en accord avec l'ambiance du projet.

A l'aide de l'expérience que Clément a accumulée après dix ans de pratique de piano et de composition musicale, nous avons utilisé le logiciel gratuit MuseScore pour composer un thème principal pour le jeu. Ce travail de composition nous a permis d'adapter précisément la musique à l'univers du jeu, en réfléchissant à la fois aux émotions à transmettre et au rôle de la musique dans l'expérience du joueur.

L'objectif des musiques d'ambiances est de créer une atmosphère lourde et pesante, en cohérence avec l'univers sombre du début du jeu. Pour cela, nous avons principalement utilisé des tonalités mineures (comme le La mineur ou le Mi mineur), qui sont souvent associées à des émotions mélancoliques, sombres ou pesantes, ce qui correspond particulièrement bien à l'ambiance du début du jeu. Ces choix musicaux permettent de renforcer le ressenti global du joueur.

Pour le thème principal, nous avons finalement choisi le sol dièse mineur (G# minor), une tonalité qui évoque des émotions profondes et intenses.

Nous avons également travaillé sur le rythme et le tempo afin d'éviter une musique trop rapide ou agressive. Un tempo relativement lent permet de renforcer la sensation de "lourdeur".

Nous avons aussi utilisé un son de piano "felt", qui donne un rendu plus feutré, intime et pesant, ce qui correspond bien à l'ambiance du jeu. Nous avons créé ce thème comme un élément central de l'identité sonore du projet, comme un thème principal. (cf. Figure 1 pour voir la partition)

Il y a également un thème musical spécial pour l'écran de mort (Game Over), il s'agit d'une dérive du thème principal dont la vitesse et la hauteur ont été diminuées ce qui permet au morceau d'avoir un son beaucoup plus tragique. Il y a également un filtre passe-bas qui étouffe les hautes fréquences donnant l'impression que la musique est jouée "sous l'eau". Ce thème spécial est donc joué automatiquement quand un joueur meurt et qu'il atteint le menu "Game Over".

Clément a également composé un thème entièrement dédié au combat contre le boss final. Contrairement au thème principal du jeu qui se voulait très sombre et pesant, ce nouveau morceau a été pensé pour apporter une dynamique plus épique et intense pour marquer le combat de fin contre le boss "Sire Radiantus Maximus".

La composition de ce second thème pour le jeu a été réalisée sur une tonalité de La majeur (avec sa relative en Fa dièse mineur). Ce choix permet de casser la monotonie du thème principal pour montrer que le joueur a accompli quelque chose et également montrer le danger que représente le boss.

Cette composition utilise également de nouveaux instruments en plus du piano d'origine. Le thème intègre désormais des lignes de violon pour accentuer le rythme et la mélodie ainsi que de la flûte basse. La flûte basse est un instrument à vent qui sonne une octave plus grave qu'une flûte traversière. Nous avons choisi d'utiliser une telle flûte spécialement pour son ton plus grave qui rappelle que le joueur se situe toujours dans un monde pesant et lourd.

L'ensemble du thème n'est pas joué pendant le combat de boss, durant le combat de boss seulement le chorus (partie principale) du thème est joué et est coupé à un moment précis pour que la répétition du thème soit fluide et sans coupures. (cf. figure 18)

La composition des musiques s'est déroulée en plusieurs étapes. Dans un premier temps, nous avons testé différentes mélodies simples au piano afin d'en trouver une cohérente avec l'univers du jeu.

Une fois une base mélodique trouvée, nous avons progressivement ajouté les accompagnements, les accords et les variations rythmiques.

MuseScore nous a permis d'expérimenter rapidement plusieurs versions d'une même composition avant de choisir celle qui correspondait le mieux à l'ambiance recherchée.

La musique dans un jeu ne doit pas uniquement être agréable à écouter, elle doit aussi accompagner l'expérience du joueur. Une musique trop dynamique pourrait créer une tension alors qu'une musique trop calme pourrait donner une impression de vide dans certaines situations.

Nous avons donc essayé de trouver un équilibre, le thème principal a été conçu pour rester suffisamment mémorable pour participer à l'identité du jeu tout en évitant une répétition trop marquée pendant de longues sessions de jeu.

Nous avons également réfléchi à la manière dont les futures zones pourraient posséder leur propre identité sonore. Certaines zones pourraient utiliser davantage de sons graves ou des rythmes plus lents afin de renforcer une sensation d'isolement tandis que d'autres pourraient utiliser des instruments plus présents pour accompagner des passages plus dynamiques. Le but final est d'obtenir une cohérence entre l'environnement visuel et sonore pour avoir une immersion optimale.

La création des effets sonores a été un travail différent de celui de la composition musicale. Contrairement aux musiques, les bruitages doivent être courts,

immédiatement reconnaissables et suffisamment clairs pour fournir une information rapide au joueur.

Nous avons donc essayé de produire des sons simples mais efficaces, capables d'indiquer certaines actions importantes sans trop surcharger l'audio global du jeu.

Nous avons aussi fait le choix de les enregistrer nous-mêmes afin de garder une cohérence avec le reste du projet. Nous avons principalement utilisé notre voix pour simuler certains sons, notamment pour les actions comme le saut ou le dash. Ce procédé nous a permis d'obtenir des résultats originaux et personnalisés.

Nous avons également enregistré nos propres bruits de pas pour les effets sonores de déplacement (course). Cela nous a permis d'ajuster précisément les sons en fonction des animations et du jeu.

Sur le plan technique, les musiques et effets sonores sont intégrés directement dans le moteur du jeu grâce aux fonctionnalités audio de Pygame.

Les musiques sont chargées au lancement du jeu puis jouées en boucle afin de maintenir une ambiance continue pendant l'exploration.

Les effets sonores sont quant à eux déclenchés dynamiquement en fonction des actions du joueur, comme les sauts, les dash, les déplacements ou les attaques.

L'intégration des musiques dans Pygame s'appuie sur le module `pygame.mixer.music` qui permet de lire des fichiers audio lourds en streaming directement depuis le disque plutôt que de les charger entièrement en mémoire, ce qui évite de ralentir et d'alourdir le jeu. Les effets sonores (SFX) sont plus courts et répétés très fréquemment donc ils utilisent la classe `pygame.mixer.Sound`. Ces fichiers sont préchargés sous forme de canaux audio au lancement du jeu. Cette séparation entre les musiques et les effets sonores est importante pour permettre la superposition sonore des deux éléments. Le joueur peut donc par exemple entendre l'effet sonore d'un dash ou d'un saut sans que cela ne coupe la musique d'ambiance qui tourne en boucle en arrière-plan.

En plus du rôle immersif des effets sonores, ils représentent également une source importante d'informations pour le joueur. Certains sons permettent de transmettre des informations sans nécessiter d'éléments visuels supplémentaires à l'écran.

Par exemple, un bruit de saut ou de dash permet immédiatement au joueur de savoir quelle action vient d'être effectuée. Tandis qu'un son lié à une attaque ennemie pourrait servir d'avertissement avant une situation dangereuse par exemple.

Nous avons donc cherché à maintenir un équilibre entre immersion et utilité afin que les effets sonores renforcent l'expérience sans devenir envahissants, répétitifs ou pénibles.

3.7. Site Web

Afin d'accompagner le projet, nous avons conçu un site web composé de plusieurs rubriques et organisé par un style css simple et coloré. Nous avons choisi des couleurs plutôt sombres comme un bleu très foncé #0a0e27 ou encore un bleu marin #1e3a8a qui représentent bien les couleurs et l'ambiance de notre jeu.

Pour chaque rubrique nous avons créé des classes pour pouvoir manipuler leur style (dimensions / couleurs) séparément depuis le fichier css afin d'obtenir un fichier structuré.

Le site est hébergé par github pages depuis le repository principal du jeu dans le dossier 'docs' situé à la racine du projet.

A chaque nouveau commit, le site se met à jour automatiquement par les actions github (ensemble d'actions automatisées par github).

Il est possible de télécharger l'ensemble des rapports de soutenance depuis le site dans l'onglet "Rapports".

Nous avons également ajouté un bouton de téléchargement directement sur la page d'accueil, permettant à l'utilisateur d'installer le jeu. Ce bouton pointe vers l'exécutable Windows hébergé sur le repository GitHub du projet. Enfin, une rubrique "Ressources" a été intégrée en bas du site, regroupant les outils utilisés lors du développement (Pygame, LDK et Git) chacun accompagné de son logo et d'un lien vers son site officiel.

3.8. Contrôles

Nous avons implémenté les contrôles de base pour le joueur, qui constituent le cœur du gameplay et permettent une prise en main rapide et intuitive du jeu. L'objectif était de proposer des commandes simples, efficaces et accessibles, tout en laissant suffisamment de précision pour les phases de plateforme (mouvement).

Le joueur peut utiliser les touches Q / D ou les flèches gauche / droite afin de se déplacer horizontalement. Ce double choix de touches permet de s'adapter aux préférences de chaque joueur, certains étant plus à l'aise avec les touches directionnelles classiques, tandis que d'autres préfèrent utiliser le clavier en configuration AZERTY.

Pour sauter, le joueur peut utiliser la touche espace ou la flèche vers le haut. Le saut est une mécanique essentielle dans un jeu de plateforme. C'est pour cela que nous avons peaufiné les paramètres de cette action afin de nous satisfaire le plus possible.

De plus, pour effectuer un dash, le joueur peut utiliser les touches shift gauche ou shift droite. Cette capacité permet au joueur de se déplacer rapidement sur une courte distance, que ce soit pour franchir des obstacles, éviter des dangers ou atteindre des zones autrement inaccessibles. Le dash permet d'enrichir les possibilités de déplacement.

En plus de ces mécaniques de base, nous avons ajouté des déplacements avancés afin d'enrichir encore plus le gameplay et de rendre les phases de plateforme plus dynamiques.

Notamment, le joueur peut effectuer un wall jump (saut contre un mur) ainsi qu'un wall slide (glissade le long d'un mur). Lorsque le personnage est en contact avec un mur en étant en l'air, il peut s'y appuyer temporairement et ralentir sa chute. Cela permet au joueur de mieux contrôler sa descente et de se repositionner plus facilement.

Le wall jump permet lui de prendre appui sur un mur pour rebondir dans la direction opposée. Cette mécanique ouvre de nouvelles possibilités en termes de level design, notamment pour atteindre des zones en hauteur ou enchaîner les wall jump dans un couloir vertical.

Une autre mécanique importante que nous avons implémentée est le coyote time. C'est une courte fenêtre de temps pendant laquelle le joueur peut encore sauter même s'il vient de quitter le sol. Cette fonctionnalité permet d'apporter

beaucoup de confort dans le jeu car sans elle beaucoup de joueurs pourraient rater leur saut en essayant de sauter au dernier moment. C'est aussi une fonctionnalité qui est présente dans une grande majorité des jeux de plateforme aujourd'hui.

L'entièreté du système de déplacement du joueur repose sur un fonctionnement basé sur la vitesse, ce qui permet d'obtenir des mouvements plus naturels et progressifs. Plutôt que de déplacer instantanément le joueur à une vitesse fixe, le jeu applique une accélération lorsque le joueur commence à se déplacer, puis fixe une décélération lorsqu'il s'arrête. Cela donne une sensation de poids et d'inertie, rendant les déplacements plus fluides et réalistes.

Ce système s'applique à la fois aux mouvements horizontaux et aux mouvements verticaux. Par exemple, la gravité (variable `PLAYER_FALL_ACCELERATION`) influence progressivement la chute du joueur, ce qui signifie qu'il accélère en tombant jusqu'à atteindre une vitesse maximale. De la même manière, certaines actions comme le saut, le dash ou le wall jump modifient directement cette vitesse.

Nous avons fait très attention aux sensations de contrôle du personnage. Dans un jeu de plateforme, les commandes doivent répondre immédiatement aux actions du joueur tout en donnant une sensation de fluidité dans les déplacements.

Un personnage trop rigide ou trop glissant peut rapidement rendre le gameplay frustrant, même si les mécaniques fonctionnent correctement d'un point de vue technique. Nous avons donc ajusté progressivement différents paramètres tels que l'accélération, la décélération, la vitesse maximale ou encore les forces appliquées lors des sauts et des dashes.

Nous avons réalisé ces ajustements avec de nombreux essais directement en jeu. Certains paramètres semblaient très bien, mais après un certain temps de jeu, on voyait qu'ils n'étaient peut-être pas aussi bien que ce que l'on pensait. Cette phase de réglage nous a permis de voir comment chaque paramètre pouvait influencer le confort du joueur.

Le joueur peut attaquer en appuyant sur la touche J ou avec le clic gauche de la souris. L'attaque a une portée limitée devant le joueur dans laquelle si on trouve un ennemi, celui-ci prendra des dégâts. Il y a également un délai entre chaque attaque pour éviter le spam et chaque ennemi ne peut être touché qu'une seule fois par attaque.

3.9. Système de sauvegarde

Afin de garantir la persistance des données entre les sessions de jeu, nous avons mis en place un système de sauvegarde automatique reposant sur le format JSON. Ce choix s'explique par la lisibilité du format, sa compatibilité avec Python via le module `json` de la bibliothèque standard, et sa légèreté pour des données de jeu simples.

L'architecture du système est écrite dans une classe dédiée, `SaveManager`, respectant ainsi le principe de séparation des responsabilités. Cette classe prend en charge trois fonctionnalités principales : la sérialisation de l'état du jeu (conversion des objets Python en texte JSON), la désérialisation lors du chargement (reconstruction des objets Python à partir du fichier JSON lu), et le déclenchement automatique de la sauvegarde à intervalle régulier.

Les données sauvegardées comprennent la position et les points de vie du joueur, l'index du niveau en cours, ainsi que des métadonnées telles que l'horodatage et le temps de jeu cumulé.

La sauvegarde est écrite dans un fichier `save.json`. Ce fichier est placé dans le dossier des données d'applications de l'utilisateur (`AppData`), et notre code s'occupe de le créer automatiquement s'il n'existe pas encore (cf. figure 16).

Ce choix d'emplacement est indispensable à cause des règles de sécurité Windows. En effet, notre installeur va placer le jeu dans le dossier `Program Files`. Ce dossier est protégé et interdit toute modification de fichier sans les droits d'administrateur. Si nous avions laissé le fichier de sauvegarde à côté de l'exécutable, Windows aurait bloqué l'écriture et le jeu aurait planté à chaque tentative de sauvegarde. En utilisant le dossier `AppData`, le jeu peut lire et écrire librement ses données sans jamais bloquer le joueur.

L'autosauvegarde est déclenchée toutes les 300 frames, soit environ toutes les cinq secondes à 60 FPS, grâce à un compteur incrémenté à chaque appel de la méthode `update()`. Le joueur peut également déclencher manuellement une sauvegarde ou un chargement via les touches F5 et F9. Ce système est volontairement limité au mode hors-ligne, le mode multijoueur nécessitant une gestion de la progression côté serveur.

3.10. Système de fin de partie

Jusqu'à présent, une partie d'Abyssal Ascension n'avait pas de vraie condition d'échec. Le joueur ne pouvait pas subir de dégâts et il ne pouvait pas infliger de dégâts. Maintenant, avec l'implémentation du système de points de vie et de combat du joueur et des ennemis, il est possible d'ajouter un vrai système de fin de partie. Nous avons donc mis en place un système de fin de partie qui gère la mort du joueur et lui permet de relancer une nouvelle partie.

En mode solo, lorsque le joueur perd la totalité de ses points de vie (en se faisant attaquer par des ennemis), un écran "GAME OVER" s'affiche. Dans cet écran, le joueur peut entendre la musique spécifique à l'écran de mort d'Abyssal Ascension, il s'agit d'une version modifiée du thème principal avec un air beaucoup plus tragique que l'original (plus d'information sur la création de cette variante du thème principal dans la section 2.6 Son & Musique). Le joueur peut alors appuyer sur la touche R pour réapparaître depuis le début de la partie. Une réapparition ne fait pas que réapparaître le joueur, l'ensemble des ennemis est également réinitialisé, de sorte que le joueur retrouve un niveau dans son état initial et puisse retenter le parcours dans des conditions identiques.

Un point important de ce système concerne son comportement en arrière-plan. Lorsque l'écran de fin est actif, seul le déplacement du joueur est interrompu : les ennemis continuent d'être mis à jour et en multijoueur, la synchronisation réseau poursuit son fonctionnement normal. Ceci est essentiel pour qu'un joueur mort ne bloque pas la partie ni n'affecte l'expérience des autres joueurs encore en vie.

En multijoueur, lorsqu'un joueur meurt, l'écran "GAME OVER" s'affiche, cependant celui-ci diffère de celui du mode solo. En multijoueur, un délai de 5 secondes est imposé avant de pouvoir réapparaître. Et lors d'une réapparition via cet écran, les ennemis ne sont pas remis à zéro. Tant qu'au moins un joueur reste en vie, la mort d'un autre joueur déclenche son propre écran de fin de partie sans interrompre le jeu pour les autres. Le joueur mort disparaît simplement de l'écran de ses coéquipiers.

Le choix du délai de 5 secondes en multijoueur avant la réapparition n'a pas été fait au hasard. Nous avons testé plusieurs valeurs différentes : un délai trop court (1 ou 2 secondes) rendait la mort presque sans conséquence et les joueurs n'avaient pas vraiment peur de mourir. À l'inverse, un délai trop long (10 ou 15 secondes) devenait frustrant. Cinq secondes nous a donc paru comme être un bon choix. De plus, l'ajout de ce délai a aussi l'avantage de créer des moments de tension intéressants dans le multijoueur. Quand un joueur meurt ses coéquipiers savent qu'ils vont devoir survivre un peu sans lui ce qui peut changer le comportement pendant les combats par exemple.

Cependant, si tous les joueurs meurent en même temps, un écran spécial “DÉFAITE TOTALE” remplace le “GAME OVER” classique. La différence n’est pas seulement visuelle, lorsque le joueur appuie sur R depuis l’écran de “DÉFAITE TOTALE”, l’ensemble des ennemis sont remis à zéro.

Cela permet de rajouter un esprit d’équipe dans la gestion des joueurs morts, par exemple, prendre des risques en étant le dernier joueur en vie (les autres joueurs sont morts et doivent attendre la fin du délai de réapparition) est sûrement une mauvaise idée car cela pourrait mener à la mort de tous les joueurs et donc à devoir recommencer de zéro.

La gestion de la fin de partie représente aussi un élément important dans l’équilibrage global du jeu. Elle ne doit pas être uniquement une condition d’échec, c’est aussi censé être un moyen pour le joueur de comprendre ses erreurs.

Dans un jeu de type Metroidvania, la mort du joueur fait partie du jeu. Elle permet de découvrir progressivement les mécaniques des ennemis et de mieux anticiper leurs comportements. Le système de réapparition a donc été fait pour encourager la progression plutôt que de tout le temps pénaliser le joueur.

En multijoueur, ce problème est encore plus complexe car la mort d’un joueur ne doit pas interrompre l’expérience des autres participants. Nous avons donc fait attention à séparer les états de chaque joueur afin que chacun puisse continuer à progresser indépendamment des autres.

3.11. Création de l'exécutable

La création de l’exécutable du jeu a nécessité quelques changements dans l’infrastructure de notre code. Premièrement nous avons dû revoir la gestion des chemins d’accès à nos ressources (assets). Durant le développement, nos chemins relatifs dépendaient du répertoire de travail de notre éditeur de code (incluant le préfixe `src/`, par exemple `src/assets/background.jpg`). Cependant, une fois compilé, l’exécutable devient le nouveau point de référence. Nous avons donc dû transformer tous les chemins dans l’ensemble de nos scripts (`launcher.py`, `ldtk_loader.py`, etc) pour qu’ils aient un comportement identique lors de l’exécution durant le développement et dans l’exécutable final.

Deuxièmement, il a fallu configurer spécifiquement notre outil de compilation (PyInstaller) pour que toutes les données externes soient intégrées dans

l'exécutable final. Par défaut, un exécutable classique ne va intégrer que le code source. Nous avons donc dû utiliser des commandes spécifiques (`--add-data`) pour forcer l'inclusion de nos dossiers importants (assets, world, ...) directement dans le dossier final. Nous avons également dû fournir l'icône d'application de notre jeu sous le format spécifique `.ico` pour PyInstaller, afin que l'exécutable final du jeu ait pour l'icône de notre jeu.

Troisièmement, nous avons adapté la structure de compilation pour contourner le comportement par défaut de PyInstaller. Nous avons dû utiliser le paramètre `--contents-directory "."` pour forcer la création d'une structure "plate" afin que l'exécutable principal puisse accéder aux ressources correctement (normalement PyInstaller met les ressources dans un dossier `"_internal"` rendant l'accès aux ressources différents).

Ensuite, nous avons utilisé Inno Setup afin de créer un faire processus d'installation pour le jeu. Nous avons commencé par renseigner les détails de notre jeu dans l'application Inno Setup (cf. figure 17). Il a ensuite fallu lier l'exécutable principal (launcher.exe) comme point d'entrée de l'application, tout en disant à l'installateur d'intégrer récursivement l'ensemble du dossier généré par PyInstaller (comprenant les dossiers assets, world et les bibliothèques). Nous avons également fourni à Inno Setup l'icône de notre jeu sous format `.ico` afin que l'exécutable d'installation du jeu ait la même icône que l'exécutable de notre jeu.

Le résultat est un fichier `.exe` d'installation. Son exécution automatise l'installation du jeu sur la machine : il extrait la structure, configure les raccourcis système (bureau, menu démarrer) et génère un utilitaire de désinstallation spécialement conçu pour Windows. Ceci nous permet de distribuer le jeu sous une forme standardisée et donc simple d'utilisation.

Cette étape de création de l'exécutable a également permis de mettre en évidence certaines différences importantes entre l'environnement de développement et une application destinée à être distribuée à d'autres utilisateurs.

Durant le développement, le programme est exécuté directement depuis l'environnement de travail et possède donc un accès immédiat aux différents fichiers utilisés par le projet. Une fois compilée, l'application devient cependant beaucoup plus indépendante et nécessite une organisation plus stricte des ressources.

Cette phase nous a donc permis de découvrir plusieurs problématiques liées à la distribution d'un logiciel, notamment la gestion des dépendances, l'organisation des fichiers et les contraintes propres au système d'exploitation Windows.

4. Réalisation

4.1. Fonctionnalités jouables actuelles

Pour cette dernière soutenance, le joueur peut lancer le jeu et choisir plusieurs options dans le menu d'accueil avec le thème musical principal du jeu en arrière-plan.

Il peut choisir entre créer une partie en solo (hors ligne), héberger une partie ou rejoindre une partie déjà hébergée ailleurs par une autre machine du même réseau.

Les joueurs peuvent donc créer une partie avec un joueur serveur, et des joueurs clients. Dans la partie multijoueur, les joueurs peuvent se voir et se déplacer en utilisant des mouvements tels que, la course (gauche / droite), le saut, le double saut, dash, le wall jump et le wall slide. Les joueurs peuvent se différencier entre eux en activant l'affichage des "nametags" depuis le menu pause. Les nametags affichent l'identifiant d'un joueur sous son personnage, par exemple : "Player 1" ou "Player 2" sachant que le serveur aura toujours le nametag "Player 1".

Les joueurs se retrouvent dans la map que nous avons créé de toute pièce et qui a été chargée avec les textures correspondantes dans le jeu. Ils peuvent changer de zone en traversant les "portes" (zones désignées permettant de changer de zone) et explorer la map.

La map contient également des IA ennemis qui patrouillent et peuvent détecter les joueurs dans leur champ de détection avant de les poursuivre et de les attaquer, les IA peuvent attaquer en continu dans une zone désignée si un joueur s'y trouve.

Les joueurs peuvent se défendre en attaquant également les IA, chaque IA peut mourir mais peut également tuer un joueur. Un joueur mort peut décider de réapparaître à partir de l'écran "GAME OVER" (le comportement de fin de partie est décrit plus précisément dans la partie 2.10 "Système de fin de partie"). Chaque joueur peut voir sa vie avec la barre de vie affichée en bas à gauche de l'écran. Quand la valeur atteint 0, le joueur meurt.

Tout type de dégât est également affiché par un court flash visuel rouge sur l'entité qui prend des dégâts.

Les joueurs peuvent donc s'aventurer dans la carte d'Abyssal Ascension. A force de monter et d'avancer dans la carte, ils vont pouvoir affronter le boss du jeu : "Sire Radiantus Maximus". Ils vont devoir coopérer ensemble pour réussir à l'affronter sans mourir malgré ses attaques puissantes et imprévisibles. Durant sa deuxième phase, il deviendra plus rapide et plus imprévisible ce qui forcera les joueurs à faire davantage attention à leur positionnement.

Même si le projet n'est pas totalement terminé, les fonctionnalités actuellement implémentées permettent déjà d'obtenir une première expérience de jeu complète. Nous sommes très satisfaits de notre travail sur ce projet.

Le joueur peut explorer plusieurs zones, interagir avec différents éléments du monde, combattre des ennemis et expérimenter avec les différentes mécaniques implémentées. La présence du multijoueur ajoute également de nombreuses possibilités avec des joueurs qui peuvent interagir sur le même monde. (cf. figures 19 et 20)

4.2. Problèmes & Solutions

Lors de la création des premiers designs du personnage principal, nous avons remarqué que le niveau de détails ne convenait pas à nos attentes pour le jeu. Nous avons donc pris la décision de passer d'animations de 32x32 pixels à 64x64. Ces animations étant dessinées à la main, nous avons alors choisi de garder un design simple et efficace afin de ne pas perdre trop de temps.

Nous avons rencontré des conflits Git au cours du développement. Cela signifie que plusieurs membres de l'équipe ont modifié les mêmes fichiers en parallèle, puis ont effectué des commits de leur côté. Lors de la mise en commun des modifications (merge), Git n'était plus capable de déterminer automatiquement quelle version du code devait être conservée.

Ces conflits nous ont obligés à intervenir manuellement pour comparer les différentes versions du code et choisir les modifications à garder, voire les combiner. Cette étape est délicate car une mauvaise résolution de conflit peut entraîner des bugs ou la suppression de certaines fonctionnalités. Cette difficulté nous a aussi permis d'apprendre à mieux utiliser et à mieux exploiter le potentiel de Git.

Lors de l'implémentation du changement de résolution, nous avons eu beaucoup de problèmes avec des parties de l'interface ou même de la carte elle-même qui devenait invisible (rendu en dehors de l'écran). Ces problèmes nous ont obligés à travailler sur la mise à l'échelle de l'interface et de la carte afin de pouvoir garder tous les éléments visibles à l'écran peu importe la résolution que le joueur choisit.

Nous avons également rencontré de nombreux bugs lors des phases de test, qui ont progressivement été corrigés au cours du développement. Certains concernaient par exemple le système de sauvegarde et le chargement des zones : dans certains cas, les ennemis chargés depuis une sauvegarde ne correspondaient pas à la bonne zone (par exemple les ennemis de la zone 1 apparaissent dans la zone 2).

D'autres problèmes étaient liés au système de réapparition du joueur. Réapparaître après une mort dans une zone autre que la première pouvait parfois faire apparaître le joueur à l'intérieur d'un mur, ce qui bloquait immédiatement la partie.

En plus des problèmes techniques, nous avons aussi rencontré des difficultés liées à la communication au sein de l'équipe. Parfois, deux membres de l'équipe avaient une vision différente de la manière d'implémenter une fonctionnalité ce qui pouvait mener à des problèmes dans le code ou à des discussions bien trop longues. Nous avons donc mis en place un système où chacun pouvait travailler de son côté sur sa partie en donnant des sortes de mises à jour au reste de l'équipe de temps en temps.

Enfin, un de nos plus gros défis techniques est arrivé lors de la séparation des joueurs dans des pièces différentes en multijoueur. Au départ, le serveur ne mettait à jour que la pièce où il se trouvait, ce qui figeait complètement les monstres dès qu'un client changeait de salle. En essayant de forcer le serveur à charger et calculer toutes les salles du monde en même temps, le jeu du serveur est tombé à 3 FPS à cause du rechargement des collisions de toutes les salles en permanence. Pour corriger ce problème, nous avons créé un système de cache au démarrage pour stocker toutes les collisions de tous les niveaux dans la mémoire. Nous avons ensuite modifié la logique du serveur pour qu'il n'active la physique et l'IA que dans les salles contenant au moins un joueur. Cette solution a permis d'éliminer les chutes de framerate tout en ayant tous les ennemis qui se déplacent et attaquent correctement.

Un autre bug important lié aux sauvegardes apparaissait lorsqu'un joueur quittait la partie en mode solo alors qu'il était mort, juste après l'exécution d'une sauvegarde automatique. Lors du rechargement de cette partie, le fichier

restaurant correctement les points de vie du personnage à zéro mais le système ne déclenchait pas la fin de partie. Le joueur se retrouvait alors dans un état d'invincibilité car il pouvait continuer à se déplacer et plus aucun ennemi ne l'attaquait. Pour corriger ce problème, nous avons ajouté une vérification de sécurité directement après le chargement de la sauvegarde. Maintenant, si le système détecte que la vie du personnage chargé est inférieure ou égale à zéro, il force immédiatement l'état de mort. Le personnage joue alors son animation de mort et l'écran Game Over apparaît.

5. Synthèse des expériences individuelles

5.1. Elyas

Au cours de ce projet, Elyas a dû réapprendre certaines compétences techniques qui s'étaient estompées avec le temps, notamment le HTML pour la création du site web de présentation. Cette remise à niveau, bien que nécessaire, a engendré une perte de temps non négligeable dans la phase de conception du site. En parallèle, il a découvert et pris en main GitHub, outil qu'il n'avait jusqu'alors jamais utilisé de manière sérieuse. La gestion de versions, le système de branches et la synchronisation du travail en équipe via des dépôts distants ont constitué un apprentissage à part entière, mais se sont rapidement révélés indispensables à la bonne organisation du projet.

Sur la partie gameplay, il a réfléchi à un schéma de contrôles simples afin de garantir un confort optimal au joueur. Le choix des touches Q et D pour le déplacement horizontal, de la barre espace pour le saut et de la touche Shift pour le dash découle d'une inspiration des standards du genre. Elyas a ensuite enrichi ce système de contrôles en implémentant un mécanisme de wall jump, permettant au joueur de rebondir contre les murs pour enchaîner les déplacements verticaux, ainsi qu'un wall slide, qui ralentit la chute du joueur lorsqu'il est plaqué contre une paroi et maintient la direction correspondante. Il a également travaillé sur l'implémentation du système d'attaque du joueur, qui a permis au jeu d'être plus complet avec l'addition du système de combat.

Il a implémenté ce système de contrôles physiques en se basant sur une gestion de la vélocité (accélération et inertie), afin de garantir la fluidité des mouvements pour le jeu. Il a développé les mécaniques avancées du wall jump et du wall slide en codant les conditions nécessaires au déclenchement de chacun.

Enfin, il a développé l'architecture complète d'un système de sauvegarde automatique. Pour cela, il a créé un nouveau fichier, `save_manager.py`,

contenant une classe `SaveManager` et ses méthodes. Cette classe gère la sérialisation de l'état du jeu vers un fichier JSON, sa désérialisation au chargement, ainsi que le déclenchement automatique de la sauvegarde à intervalle régulier. Il a résolu un problème majeur lié aux restrictions de sécurité de Windows en déplaçant l'écriture du fichier `save.json` du répertoire local vers le dossier utilisateur `%AppData%` ce qui a permis au jeu de s'exécuter et de sauvegarder automatiquement toutes les 300 frames sans avoir besoin des droits d'administrateur.

L'une des plus grandes satisfactions d'Elyas est la finalisation du système de déplacement et de combat. Voir le personnage répondre instantanément aux commandes, enchaîner un dash et un wall jump de manière fluide est quelque chose dont il est fier. De plus, le fait de corriger le problème des privilèges Windows pour l'écriture du fichier de sauvegarde a également été une expérience dont il est content.

Par contre, pour Elyas, la charge de travail initiale liée à la création du site web n'a pas été quelque chose qu'il a particulièrement apprécié. Il avait oublié la plupart de ses connaissances en HTML et en CSS, ce qui lui a nécessité de les revoir.

5.2. Jonathan

Jonathan a pu grandement participer au début de la création du jeu. En effet, lors de son année de terminale, il a créé un jeu vidéo dans le cadre d'un projet et il a donc une petite expérience sur le sujet. Il a ainsi pu expliquer à l'ensemble de l'équipe comment bien structurer leur code ou encore les étapes à suivre lors du développement.

Pour les aider davantage (hormis la création de décors et d'animations), il a appris à corriger les erreurs qui rendent notre code inopérant. Cela peut être qualifié de debugging. Il a également appris le fonctionnement d'une map et des hitbox.

En tant que responsable de la direction artistique, il a géré la transition graphique du format 32x32 pixels au 64x64 sur Piskel, augmentant énormément le niveau de détail du personnage.

Depuis la première soutenance, les animations ont grandement gagné en qualités. Chaque mouvement spécifique du joueur a sa propre animation ce qui rend le gameplay actuel beaucoup plus fluide. L'ajout des IA ainsi que de leur design rendent également la partie beaucoup plus vivante et permettent de se

sentir dans un vrai environnement de jeu vidéo, bien que le fond ne soit pas encore présent pour plonger le joueur dans l'ambiance convenue pour Abyssal Ascension. Jonathan a pu découvrir lors de ses essais que la texture des monstres était importante. En effet, un mauvais mélange de couleur fait perdre toute crédibilité à une créature hostile, et par conséquent ses premiers essais de texture étaient un échec. Les couleurs plus sombres qu'il a choisies iront de paire avec la création du fond, ce qui fera en sorte de garder un équilibre cohérent dans les graphismes du jeu.

Pour assurer l'intégration dans le jeu de ses sprites, il a mis en place une nomenclature stricte des fichiers .png (par exemple : 0-Running-0) permettant au code de charger dynamiquement la bonne texture selon l'état du joueur.

L'unique problème qu'il trouve à la direction artistique est que le travail reste un peu toujours le même, et par conséquent il s'est intéressé au comportement IA afin d'aider toute l'équipe dans la création de ces dernières.

Pour Jonathan, la plus grande fierté est de voir à quel point le jeu est devenu plaisant à regarder. Voir ses propres animations tourner à 60 fps en jeu était vraiment cool. Enfin, le fait de passer un peu de temps sur le code de l'IA avec Frédéric lui a permis de changer de domaine et de faire autre chose que du dessin pur.

Du côté des difficultés, Jonathan a pas mal bloqué sur les couleurs des monstres au début. Ses premiers essais ne collaient pas du tout avec l'ambiance sombre de la grotte, ce qui était assez frustrant. De plus, le pixel art demande de dessiner les sprites image par image et cette répétition a parfois été lourde et fatigante.

5.3. Frédéric

Cette année de travail intensif sur le développement du projet Abyssal Ascension a permis à Frédéric de découvrir et d'approfondir les piliers fondamentaux du world building et de la création d'univers (world building) et de la conception de niveaux (level design) particulièrement appliqués aux exigences très strictes du genre Metroidvania. Cette expérience a été cruciale pour faire évoluer sa manière de penser le jeu vidéo. Il a appris à ne plus seulement envisager l'environnement sous un angle purement esthétique, mais à adopter la posture d'un concepteur technique. Désormais, chaque élément du décor doit être fonctionnel pour l'expérience de navigation du joueur et techniquement compatible avec les contraintes du code source.

Frédéric s'est notamment spécialisé dans l'utilisation de l'outil LDtk (Level Design ToolKit), un éditeur externe indispensable pour structurer les mondes interconnectés. Grâce à ce logiciel, il a compris comment un niveau est construit, mais également maîtrisé la gestion des différentes couches graphiques (layers), allant du placement des tuiles (tilesets) de premier plan à la définition des décors de fond, tout en intégrant les zones de collision essentielles à la jouabilité. Ce processus inclut également la mise en place complexe du système de "portes", assurant des transitions fluides et logiques entre les différentes zones du jeu.

Dans le cadre du développement d'Abyssal Ascension, Frédéric a approfondi les principes fondamentaux régissant le comportement des entités hostiles dans un environnement 2D. Bien que l'intelligence artificielle actuelle repose sur des mécaniques simplifiées, sa conception a nécessité une réflexion structurée sur la gestion des états. Ce système permet à l'ennemi de basculer entre des phases distinctes: patrouille, alerte, poursuite et attaque, garantissant une réactivité cohérente face aux actions du joueur.

L'une des contributions majeures de Frédéric a été l'implémentation d'un système de patrouille autonome. Ce mécanisme permet aux ennemis de naviguer entre des bornes définies tout en intégrant une détection active de l'environnement. L'entité est capable d'identifier les obstacles latéraux et de modifier sa trajectoire dynamiquement, ce qui évite les comportements erratiques contre les parois du niveau.

Frédéric a également travaillé sur l'optimisation du champ de vision de l'IA, introduisant des portées horizontales et verticales spécifiques pour déclencher l'état d'alerte uniquement lorsque le joueur est visible. Cette approche pluridisciplinaire, combinant rigueur algorithmique et level design via l'outil LDtk, lui permet désormais de concevoir des rencontres plus complexes et stimulantes, posant les bases pour de potentiels futurs ennemis aux comportements et mécaniques plus élaborés.

Frédéric a adoré prendre en main le logiciel LDtk et créer toute la carte du jeu. Réussir à connecter les salles entre elles avec le système de portes et voir que les transitions devenaient fluides a été un grand soulagement. Développer l'IA pour qu'elle passe d'une simple patrouille à une vraie machine à état a aussi été un moment satisfaisant.

Sa principale source de frustration a été de gérer la gravité lors des changements de niveaux verticaux. Le joueur retombait tout le temps dans la porte, et trouver la bonne astuce pour corriger cela a pris beaucoup de temps.

Enfin, les bugs de dernière minute lors des tests, comme le fait de réapparaître bloqué à l'intérieur d'un mur après être mort, ont été assez stressants à régler.

5.4. Clément

Après cette année à travailler sur le projet, Clément a appris énormément de choses sur la partie multijoueur des jeux vidéo en LAN (Local Area Network). En effet, il a dû apprendre les bases de la librairie python socket afin de transmettre des données entre deux machines du même réseau. Cela lui a permis de mieux comprendre comment les jeux multijoueur gèrent la communication entre plusieurs joueurs, notamment l'envoi et la réception d'informations comme les positions, les actions ou encore les états du jeu.

Cependant, cette partie du projet a également apporté son lot de difficultés. Par exemple, nous avons rencontré des problèmes liés à l'utilisation du protocole TCP. Contrairement à ce que l'on pourrait penser, TCP ne garantit pas que les messages envoyés soient reçus en un seul morceau. Il peut arriver qu'un message soit découpé en plusieurs parties lors de la transmission, ou au contraire que plusieurs messages soient regroupés ensemble à la réception. Cela a obligé Clément à réfléchir à une manière de reconstruire correctement les données reçues.

D'autres problématiques que Clément a dû résoudre concernaient le comportement des ennemis en multijoueur ainsi que la gestion d'états importants des joueurs, comme leur état de vie ou de mort. Par exemple, il a fallu réfléchir à des situations particulières : que se passe-t-il si le serveur meurt ? Comment permettre à la partie de continuer même lorsque le joueur hébergeant la partie est éliminé ?

La résolution de ces problématiques a permis à Clément d'approfondir ses compétences dans le développement multijoueur ainsi que dans la recherche de solutions capables de satisfaire à la fois les joueurs et les contraintes de performance.

Cette année de projet lui a également permis de travailler sur la théorie musicale et la composition de musique. Il a dû réfléchir aux choix de tonalités, d'instruments et d'ambiances afin que la musique corresponde à l'univers du jeu. La composition des deux thèmes d'Abyssal Ascension (thème principal et thème de combat de boss) lui ont pris beaucoup de temps mais nous sommes très satisfaits du résultat obtenu.

En même temps, nous avons cherché différentes solutions pour produire nous-mêmes des effets sonores, ce qui nous a demandé de faire preuve de créativité.

Ce projet lui a également appris à chercher des solutions face aux nombreux problèmes rencontrés durant le développement. Que ce soit des bugs techniques, des difficultés de conception ou des contraintes liées au travail en groupe. Cela nous a aussi appris à mieux travailler en équipe, à communiquer et à nous mettre d'accord sur les différentes fonctionnalités et décisions à prendre pour le projet.

Enfin, cette expérience lui a permis de mieux comprendre l'ensemble des étapes de création d'un jeu vidéo, des premières idées à leur implémentation.

Pour Clément, le meilleur moment a été de voir deux PC se connecter en LAN et se synchroniser correctement à chaque image. Voir les mouvements et les animations des autres joueurs s'afficher en temps réel sans bug a été le défi le plus plaisant et le plus réussi de son année. En plus du réseau, composer le thème principal au piano sur MuseScore et le voir s'intégrer parfaitement à l'ambiance du jeu a été très sympa.

Le réseau a quand même apporté beaucoup de problèmes au projet. Gérer le protocole TCP qui coupait les messages JSON en morceaux et faisait planter le jeu a demandé des heures de debugging. Essayer de coder tous les cas particuliers, comme la mort de tous les joueurs en même temps, a pris beaucoup de temps à implémenter.

6. Pr vision

6.1. Accord avec le planning

T�ches	Progression actuelle soutenance 2	Progression vis�e soutenance finale
World building	100%	100%
Character Design	100%	100%
Musique	100%	100%
IA	100%	100%
Contr�les joueurs	100%	100%
FX	100%	100%
Physique et collisions	100%	100%
Playtest	100%	100%
Interface / UI	100%	100%
Optimisation	100%	100%
Multijoueur	100%	100%
Syst�me de sauvegarde	100%	100%
Site web	100%	100%

% = en retard

% = dans les temps

% = en avance

Les prévisions qui vont suivre correspondent à ce que nous aimerions rajouter ou modifier sur le projet si nous continuons à travailler dessus après la soutenance finale.

6.2. World Design

Pour le world design, on devra ajouter de nouvelles zones et également ajouter plus de détails aux zones actuelles si possible.

Nous aimerions également pouvoir ajouter de nouvelles zones avec un aspect totalement différent, comme si on changeait de paysages, par exemple à la fin du premier boss.

Les nouvelles zones devront contenir des mécaniques que nous avons récemment ajoutées telles que le wall jump et le dash. Ces zones ne devront être accessibles qu'en utilisant ces capacités afin d'avoir une map un minimum exigeante.

6.3. IA

Pour l'IA, nous aimerions bien ajouter de nouveaux états, par exemple la fuite de l'IA si elle n'a plus beaucoup de vie, ou alors des attaques groupées entre IA si possible ou même les IA qui bloquent le passage vers une autre zone en se mettant devant la porte de transition.

On aimerait également ajouter plusieurs types différents d'ennemis, par exemple des ennemis qui attaquent à distance ou par la terre (comme des taupes). Nous aimerions également ajouter des ennemis volants qui demanderont donc au joueur de sauter et d'attaquer en même temps pour pouvoir leur infliger des dégâts.

6.4. Multijoueur

Pour le multijoueur, nous aimerions bien ajouter un système de fluide de déconnexion. Pour l'instant si un joueur se déconnecte, aucun message ne transmet aux autres joueurs que c'est le cas. De plus, si c'est le serveur qui se déconnecte, cela fera instantanément quitter le jeu aux clients qui étaient connectés.

Une fois ces fonctionnalités intégrées, nous aimerions bien avoir un système de pseudo. Chaque joueur en entrant dans une partie multijoueur pourra choisir un pseudo pour que les autres joueurs puissent l'identifier plus facilement.

6.5. Menus & Interface utilisateurs

Pour les menus et l'interface utilisateurs, nous aimerions avoir une interface globalement plus cohérente. Nous aimerions avoir notre propre police d'écriture pour le jeu afin d'avoir une direction artistique encore plus forte.

Nous aimerions aussi avoir des effets sonores de menu pour signaler quand un utilisateur clique sur un bouton et change la valeur d'un paramètre.

Nous prévoyons aussi d'implémenter le support complet de la souris pour la navigation. Actuellement, l'interface utilisateur repose uniquement sur l'utilisation des flèches du clavier pour sélectionner les options et de la touche Entrée pour valider. L'objectif serait d'intégrer la détection du curseur et des clics. Techniquement, cela demandera d'ajouter une vérification des collisions entre les coordonnées de la souris et les rectangles contenant les différentes options des menus.

6.6. Textures & Animations

L'objectif est de rendre chaque animation plus fluide en lui ajoutant 2 à 3 sprites.

Nous aimerions aussi nous occuper de graphismes spécifiques à certains menus, comme le menu pause, l'interface de mort (Game Over). Dans ces menus nous aimerions que la direction artistique du jeu se ressente plus, car nous trouvons qu'actuellement ils restent plutôt simples.

Si possible, ajouter des cinématiques et des animations spécifiques à chaque ennemi.

6.7. Son & Musique

Pour les sons et les musiques, nous aimerions ajouter un système de musique dynamique capable d'évoluer en fonction de la situation du joueur.

Par exemple, certaines couches instrumentales pourraient apparaître lors des combats ou disparaître durant les phases d'exploration calme.

Ce type de système permettrait de renforcer encore davantage l'immersion du joueur et de rendre l'ambiance sonore plus vivante.

Nous aimerions également composer plusieurs autres musiques originales pour d'autres zones et contextes (dialogue, nouvelles zones, animation...) et des effets sonores spécifiques à certains ennemis, comme un rugissement pour le boss ou des sons d'attaque et de mort pour les ennemis.

6.8. Site Web

Pour le site web, il faudra rajouter du contenu tel que les captures d'écrans du jeu, un lien d'installation et un guide d'installation.

Il faudra également le rendre plus élégant sur interfaces mobiles (en ce moment, l'interface est légèrement endommagé quand ouverte sur téléphone portable).

Nous aimerions également rendre le style du site web plus cohérent avec le jeu. Peut-être même utiliser des graphismes du jeu à l'intérieur du site web, cela pourrait rendre le site comme une extension du jeu.

6.9. Contrôles

Pour les contrôles, nous aimerions ajouter un système de combo dans le combat, celui-ci demanderait par exemple au joueur d'appuyer sur des touches spécifiques avec un timing spécifique pour effectuer des combos ou attaques différentes.

Nous aimerions également ajouter le support des manettes pour jouer au jeu car nous pensons que cela pourrait être une fonctionnalité que beaucoup de personnes pourraient apprécier.

6.10. Système de sauvegarde

Pour le système de sauvegarde, nous aimerions bien le rendre plus avancé en ajoutant un système de chargement de sauvegarde volontaire (actuellement le jeu charge automatiquement la dernière sauvegarde), cela permettrait au joueur de créer plusieurs sauvegardes et d'en charger une spécifiquement.

Nous aimerions également ajouter un système permettant de gérer ses sauvegardes, par exemple de leur donner un nom, de les supprimer ou bien même de les partager avec d'autres machines ou personnes.

6.11. Système de fin de partie

Pour le système de fin de partie, nous aimerions ajouter des statistiques concernant le joueur ou bien l'état du monde ou de l'histoire au moment de sa mort. Par exemple : on pourrait afficher le nombre d'ennemis tués, le nombre zones explorées, le moment de l'histoire où il se trouvait, etc.

Nous aimerions également ajouter d'autres options dans le menu de Game Over comme le fait de pouvoir revenir au menu principal mais aussi de quitter directement le jeu.

Nous envisageons également de lier le système de fin de partie à des mécaniques de progression. Par exemple, la mort du joueur pourrait laisser une trace visuelle ou un indicateur à l'endroit exact de sa mort lors de la tentative précédente. Techniquement, cela demandera d'ajouter une liste de coordonnées historiques dans notre structure de fichier save.json pour que le script de chargement puisse générer ces "points d'intérêts" à chaque réinitialisation du monde.

7. Conclusion

Notre travail depuis le début de ce projet s'est réalisé sans accroc et les objectifs fixés ont été respectés dans leur ensemble malgré la charge de travail.

Nous avons tous bien travaillé au sein du groupe et de l'aide est apportée dès que nécessaire si un des membres de l'équipe a des soucis pendant son travail.

Ce projet nous a permis d'en apprendre plus sur l'ensemble des étapes de développement d'un jeu vidéo. Nous avons pu voir que la création d'un jeu ne se fait pas seulement sur l'implémentation de fonctionnalités, mais aussi sur la cohérence globale entre les différents systèmes du jeu.

En plus de l'aspect technique, ce projet nous a montré à quel point il est important de planifier les tâches de l'équipe à l'avance et de communiquer en permanence entre les membres de l'équipe. Nous avons rapidement compris que le développement d'un jeu vidéo ne se résume pas à coder chacun de notre côté car toutes nos tâches sont liées. Par exemple, l'implémentation du chargement des niveaux dépend directement du level design. Cela nous a donc appris à coordonner nos différentes contributions pour pouvoir obtenir un jeu cohérent. Ce projet jeu vidéo nous a donc appris à corriger des problèmes techniques et à travailler efficacement en groupe.

Nous sommes très satisfaits de notre travail de groupe sur ce projet jeu vidéo.

8. Annexes

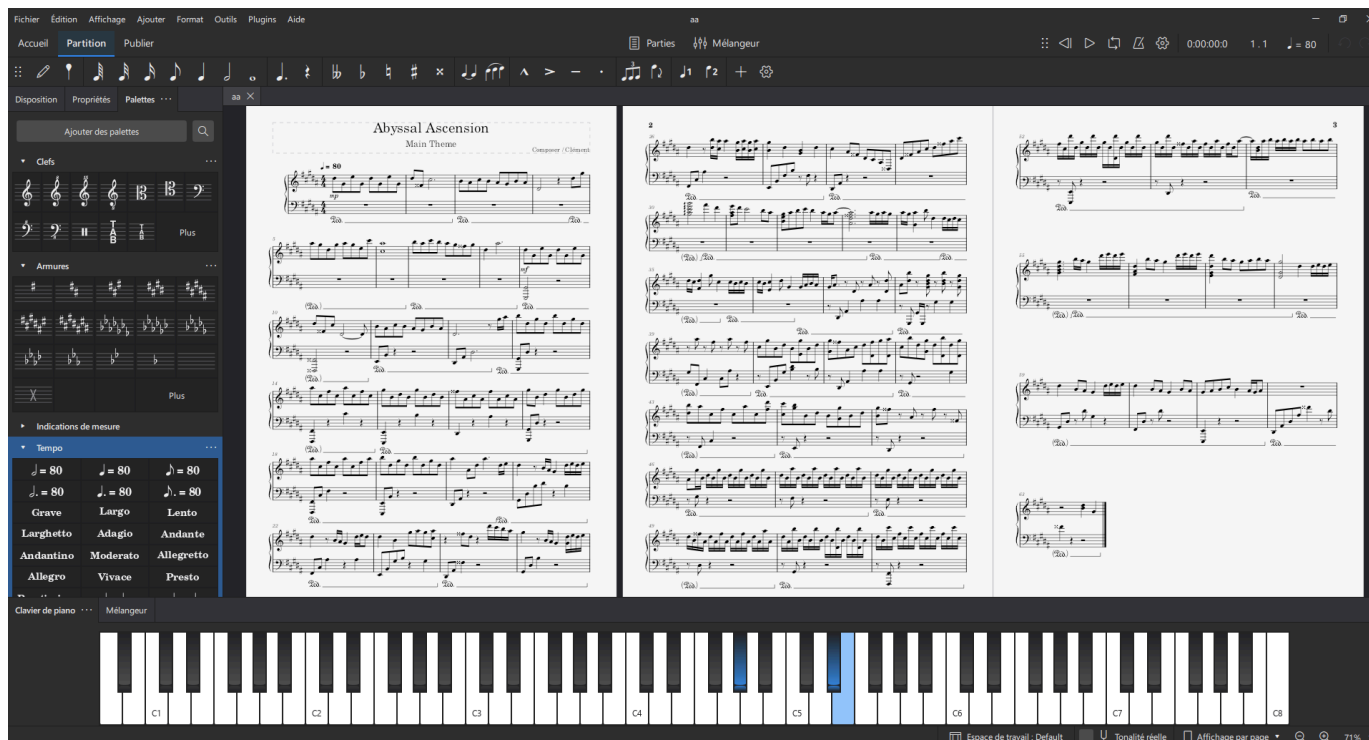


Figure 1 : Partition de musique du thème principal composée sur MuseScore.



Figure 2 : Première zone jouable

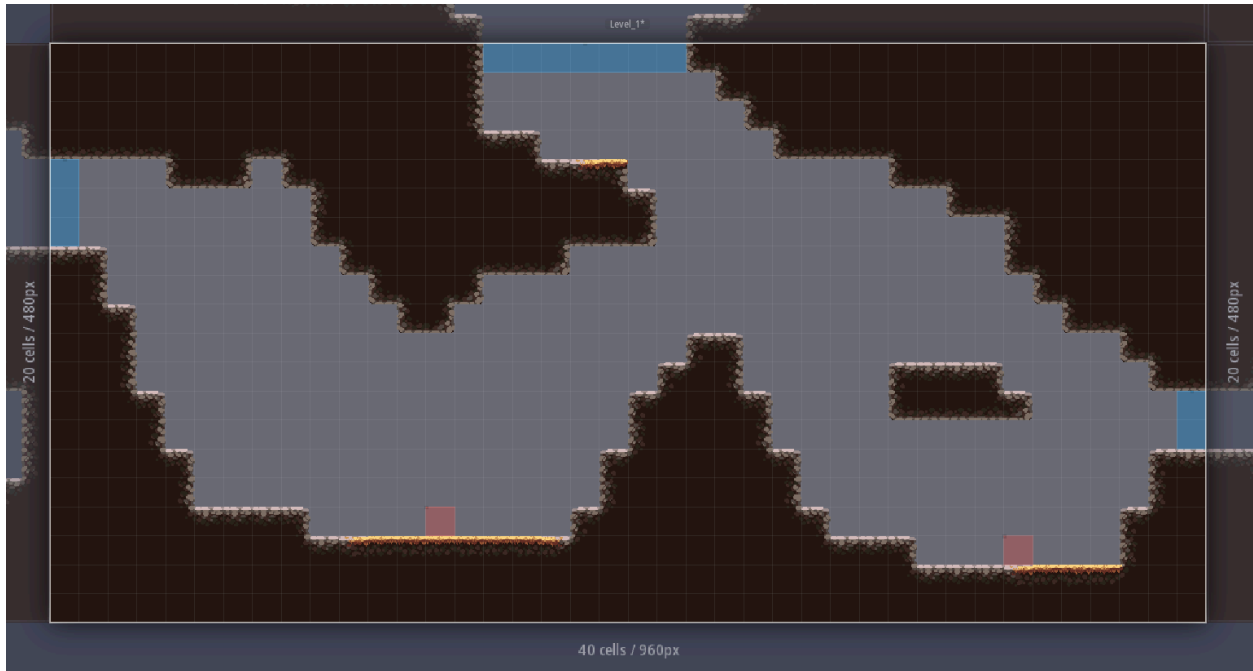


Figure 3 : 2e zone jouable

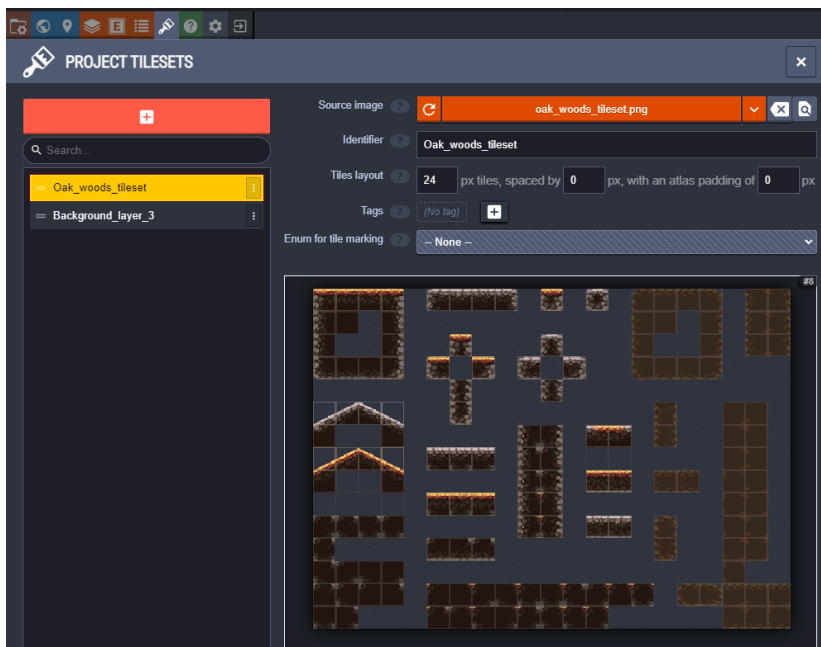


Figure 4 : Interface de configuration des tilesets sous LDtk

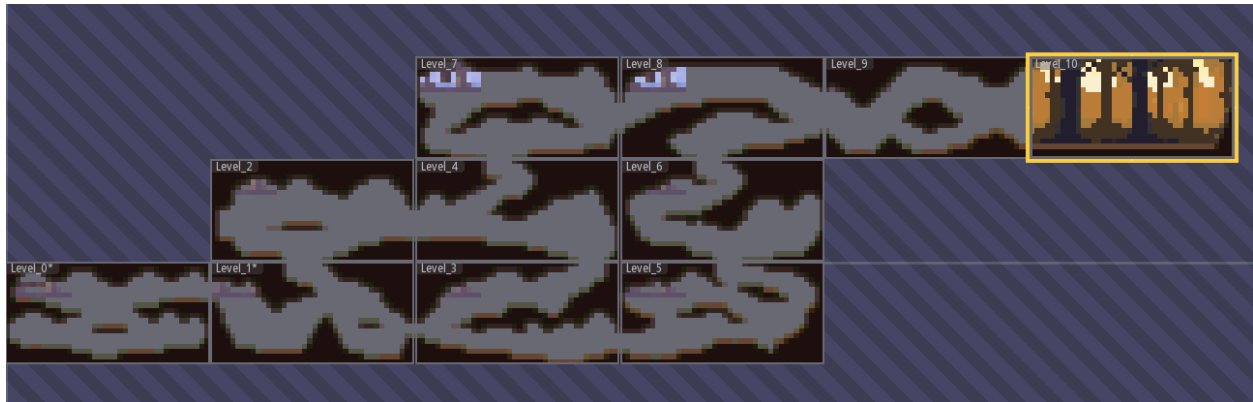


Figure 5 : Map globale du jeu sur LDK.

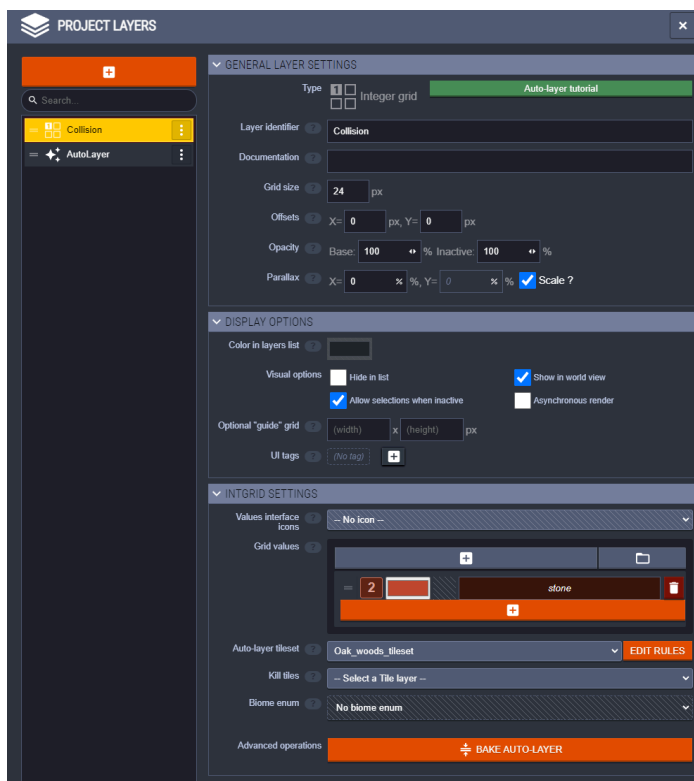


Figure 6 : Interface de gestion du World Building et des collisions

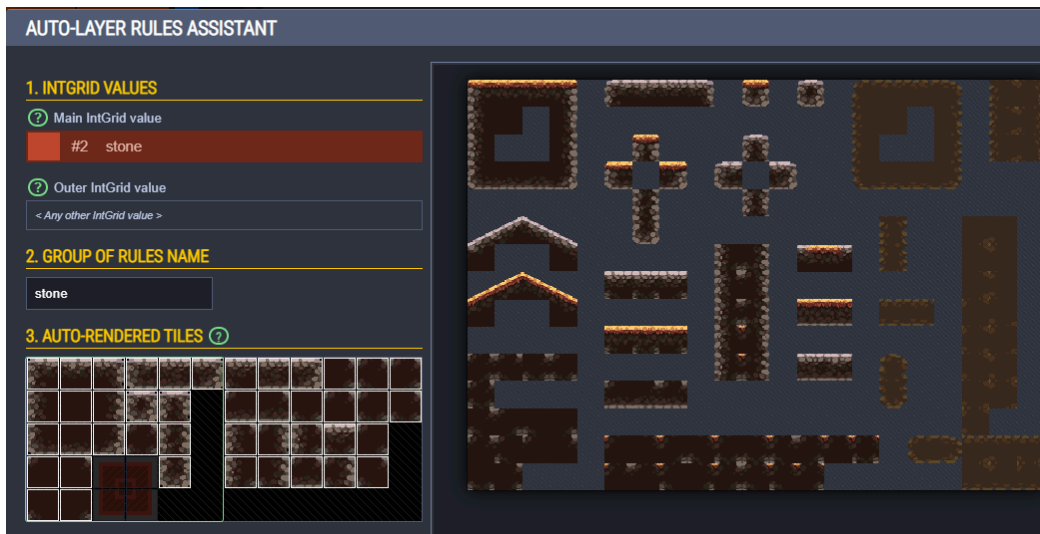


Figure 7 : Interface de configuration des motifs pour le rendu automatique du décor

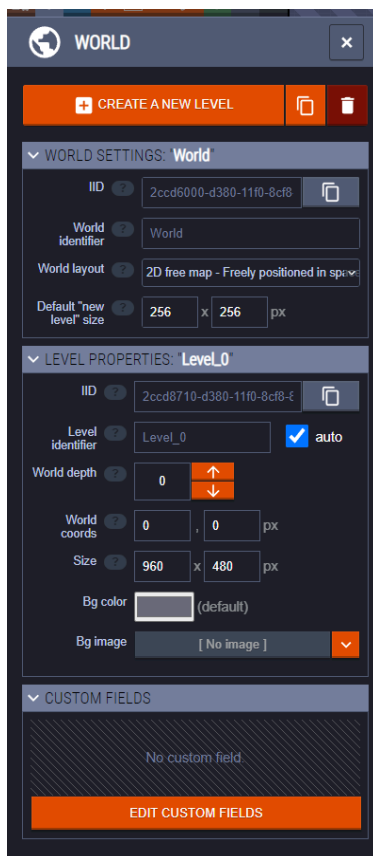


Figure 8 : Configuration des paramètres globaux du monde et du niveau dans LDtk

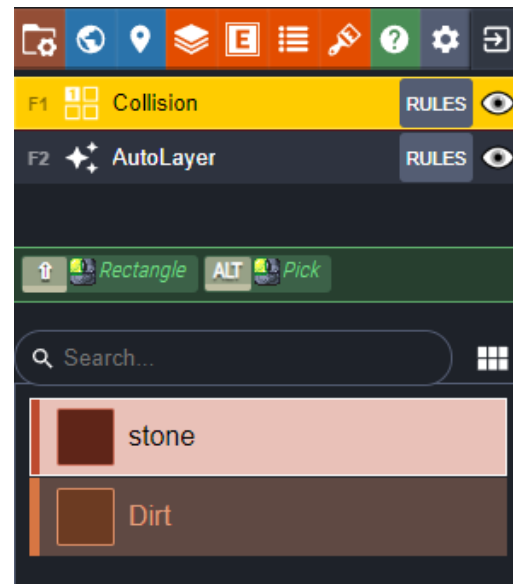


Figure 9 : Interface de sélection des matériaux pour le world building.



Figure 10 : Création d'un sprite pour le personnage principal.



Figure 11 : Création d'une animation de course en enchaînant rapidement plusieurs sprites afin de créer du mouvement.

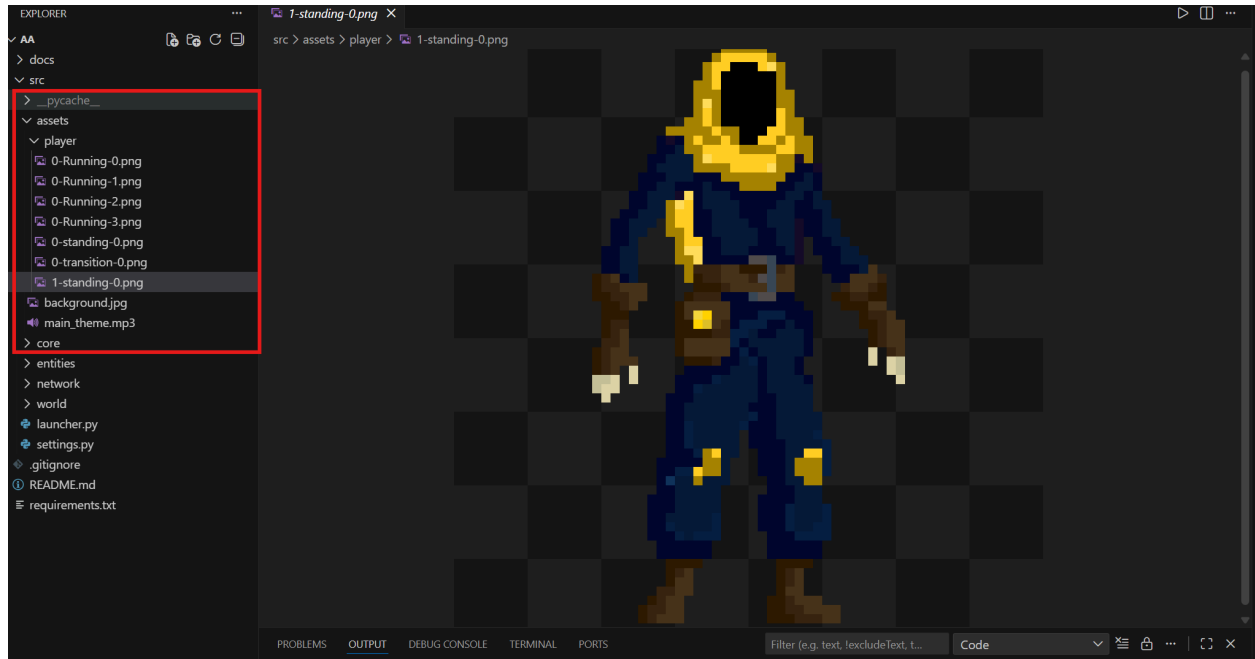


Figure 12 : Vue de l'organisation des fichiers pour les animations dans l'éditeur.



Figure 13 : Player 1

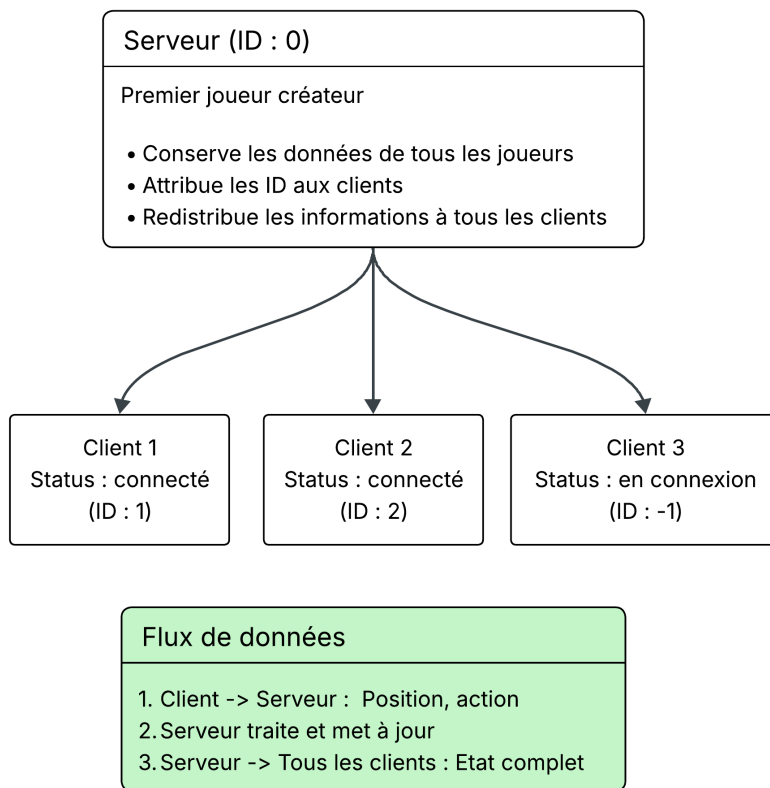


Figure 14 : Schéma représentatif du système multijoueur client-serveur

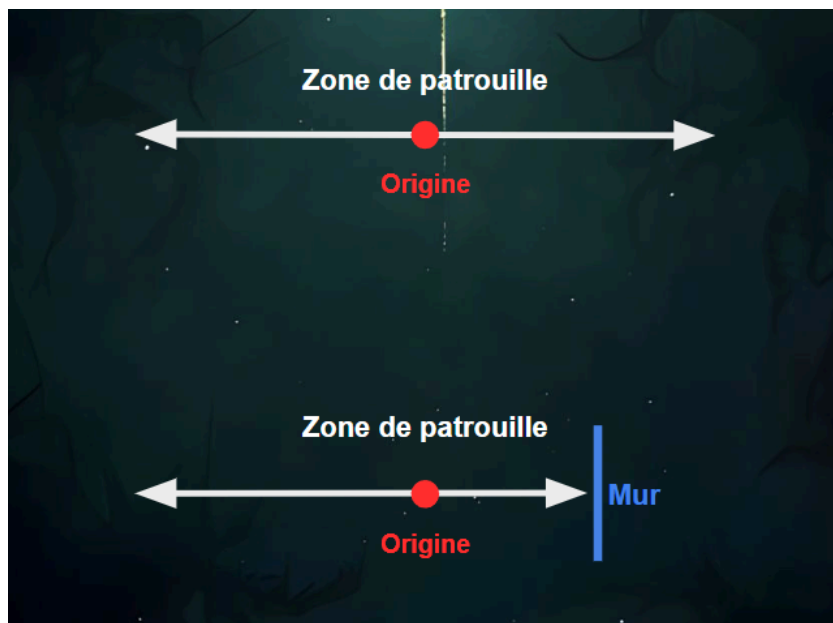
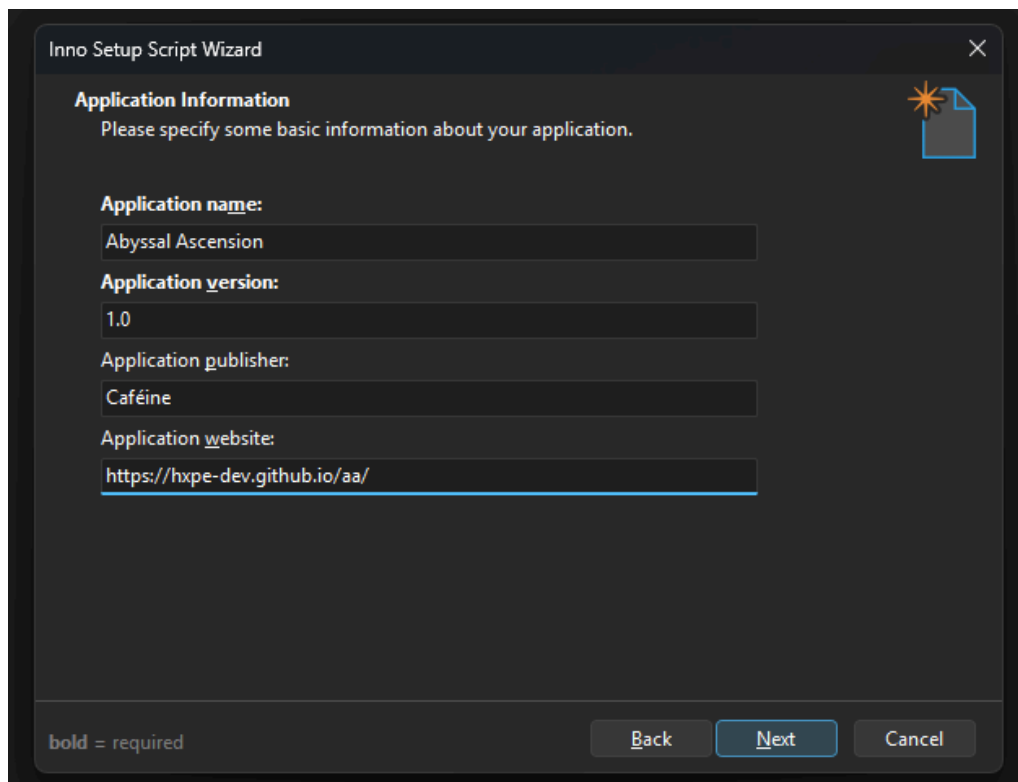


Figure 15 : Schéma de la patrouille de l'IA

```
{
  "player": {
    "x": 946.8236403199999,
    "y": 117,
    "level": 1,
    "health": 100
  },
  "meta": {
    "timestamp": "2026-03-18T17:32:13",
    "playtime_seconds": 93
  }
}
```

Figure 16: Exemple de fichier save.json



The image shows the 'Inno Setup Script Wizard' dialog box, specifically the 'Application Information' step. The window has a title bar with standard macOS window controls (red, yellow, green buttons) and a close button (X). The main content area is titled 'Application Information' and includes the instruction 'Please specify some basic information about your application.' Below this, there are four labeled text input fields: 'Application name:' with the value 'Abyssal Ascension', 'Application version:' with the value '1.0', 'Application publisher:' with the value 'Caféine', and 'Application website:' with the value 'https://hxpe-dev.github.io/aa/'. A small icon of a document with a star is located to the right of the input fields. At the bottom of the dialog, there is a legend indicating 'bold = required', and three buttons: 'Back', 'Next' (which is highlighted with a blue border), and 'Cancel'.

Inno Setup Script Wizard

Application Information
Please specify some basic information about your application.

Application name:
Abyssal Ascension

Application version:
1.0

Application publisher:
Caféine

Application website:
<https://hxpe-dev.github.io/aa/>

bold = required

Back Next Cancel

Figure 17: Renseignement des informations dans Inno Setup

Boss loop

Abyssal Ascension Composed by Clément VASSAUX

Violin

Bass Flute

Piano

3

Vln.

B. Fl.

Pno.

5

Vln.

B. Fl.

Pno.

Figure 18: Première page de la partition de musique du boss

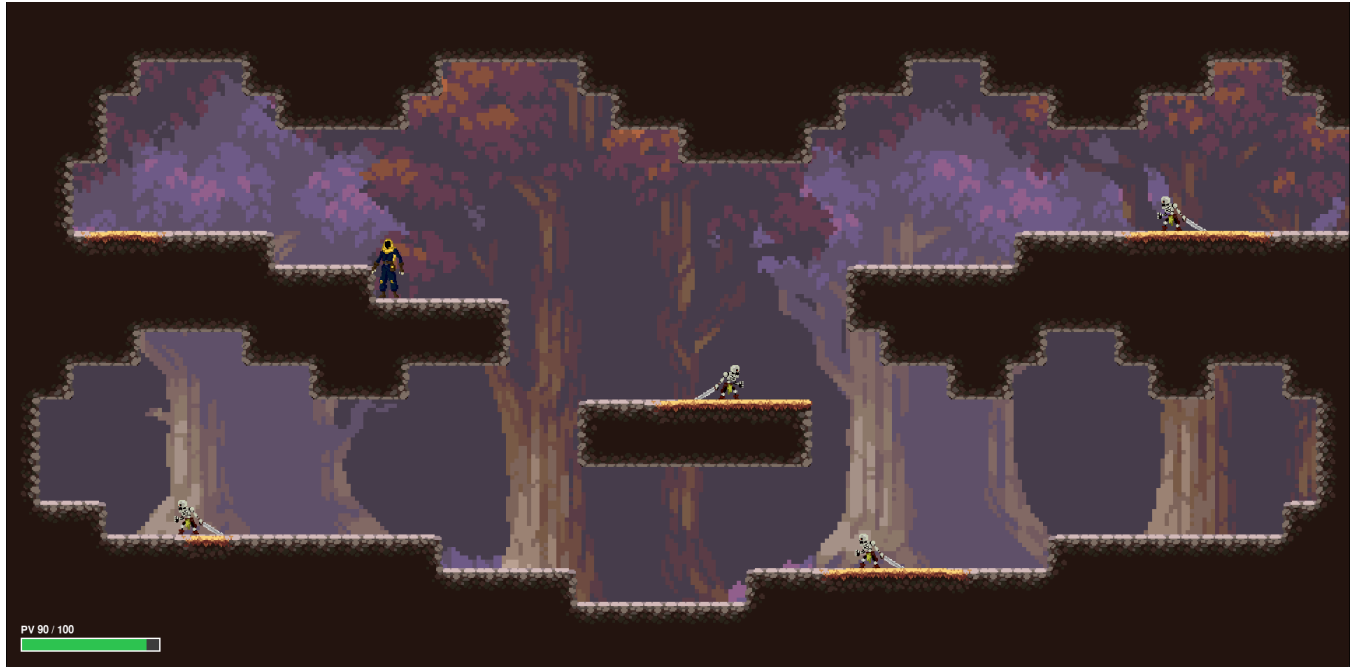


Figure 19: Première salle du jeu en mode solo.

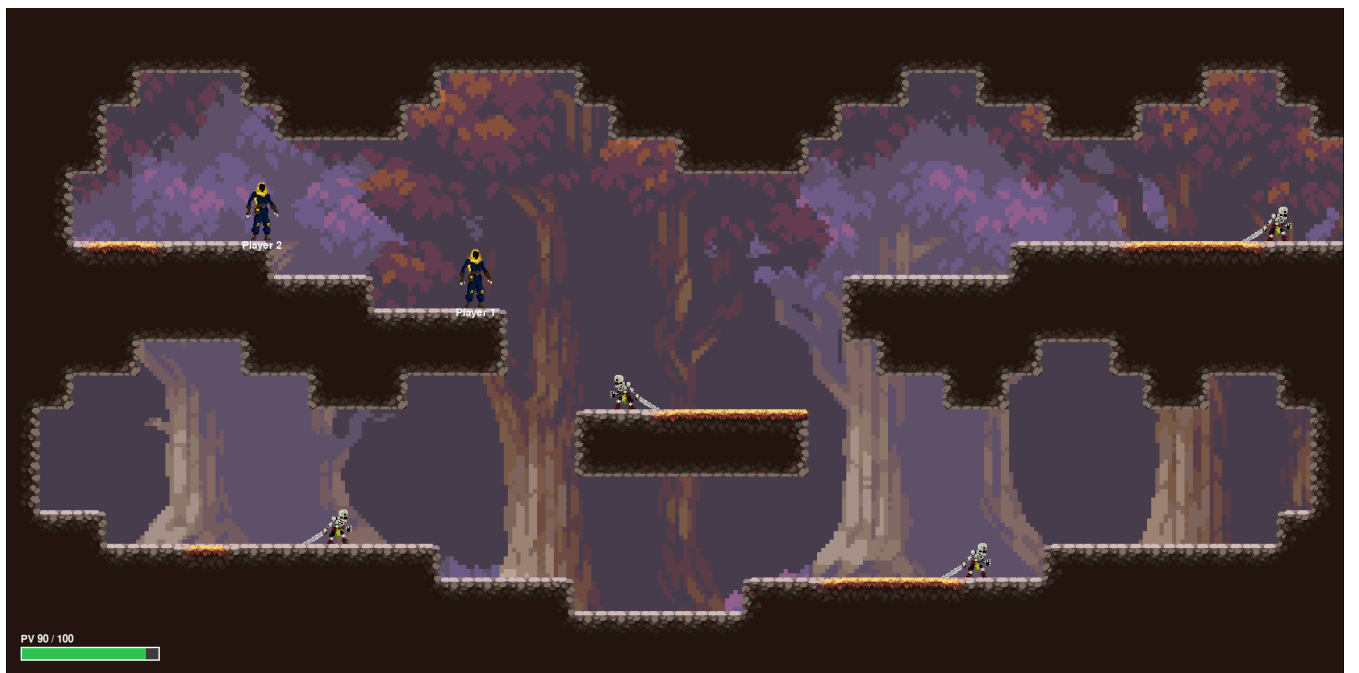


Figure 20: Première salle du jeu en mode multijoueur.